Estudo de Python e dados II

Manipulação de dados: Preparação de dados, dados ausentes e Tidy data

Sergio Pedro Rodrigues Oliveira

13 março 2026

SUMÁRIO

1 Objetivo **1**

2 Preparação dos dados **2**

2.1 Introdução . . . 2
Mapa conceitual . . . 2
Objetivos . . . 2
2.2 Tidy data . . . 3
2.2.1 Combinando conjuntos de dados . . . 4
2.3 Concatenação . . . 5
2.3.1 Adicionando linhas . . . 5
2.3.1.1 Concatenar `Series` . . . 8
2.3.1.2 Ignorando o índice . . . 11
2.3.2 Adicionando colunas . . . 12
Concatenando colunas - axis . . . 12
Subconjuntos colunas . . . 12
Adicionar uma única coluna . . . 13
Reiniciando índices colunas . . . 14
2.3.3 Concatenação com índices diferentes . . . 15
2.3.3.1 Concatenando linhas com colunas diferentes . . . 15
2.3.3.2 Concatenando colunas com linhas diferentes . . . 18
2.4 Combinando vários conjuntos de dados . . . 20
2.4.1 `merge` um a um (one-to-one) . . . 24
2.4.2 `merge` de muitos para um (many-to-one) . . . 25
2.4.3 `merge` de muitos para muitos (many-to-many) . . . 26
2.4.4 Resumo . . . 29
2.5 Conclusão preparação dos dados . . . 31

3 Dados Ausentes **32**

3.1 Introdução . . . 32
Mapa conceitual . . . 32
Objetivos . . . 32
3.2 O que é um valor `NaN`? . . . 33
3.3 De onde vêm os valores ausentes? . . . 35
3.3.1 Carga de dados . . . 35
`na_values` . . . 36
`keep_default_na` . . . 37
`na_filter` . . . 38
3.3.2 Dados combinados . . . 39
3.3.3 Valores de entrada do usuário . . . 41
3.3.4 Reindexação . . . 43

3.4 Trabalhando com dados ausentes . . . 46
3.4.1 Encontrando e contando dados ausentes . . . 46
Linhas . . . 46
Colunas . . . 48
3.4.2 Limpando dados ausentes . . . 50
3.4.2.1 Recodificar/substituir . . . 51
3.4.2.2 Preenchimento para a frente (*forward*) . . . 52
3.4.2.3 Preenchimento para trás (*backward*) . . . 53
3.4.2.4 Interpolação . . . 54
3.4.2.5 Descartando valores ausentes . . . 55
3.4.3 Cálculos com dados ausentes . . . 57
3.5 Conclusão dados ausentes . . . 59

4 *Tidy data* (dados organizados) 60
4.1 Introdução . . . 60
Mapa conceitual . . . 60
Objetivos . . . 61
4.2 Coluna contêm valores, e não variáveis . . . 62
4.2.1 Mantendo uma coluna fixa . . . 62
4.2.2 Mantendo várias colunas fixas . . . 67
4.3 Colunas contendo diversas variáveis . . . 69
4.3.1 Separar e adicionar colunas individualmente (método simples) . . . 71
4.3.2 Separar e combinar em um único passo (método simples) . . . 74
4.3.3 Separar e combinar em um único passo (método mais complicado) . . . 76
4.4 Variáveis tanto em linhas quanto em colunas . . . 79
O Problema do "Destaque" (O Índice) . . . 84
A Solução: `reset_index()` (A Reindexação) . . . 84
4.5 Várias unidades de observação em uma tabela (normalização) . . . 85
Revisão normalização de tabelas banco de dados . . . 92
4.6 Unidade de observação em várias tabelas . . . 93
4.6.1 Carregando vários arquivos usando um laço . . . 99
4.6.2 Carregando vários arquivos usando uma `list comprehension` . . . 99
4.7 Conclusão *Tidy data* . . . 100

LISTA DE FIGURAS

1 Preparação de dados, principais tópicos. 2
2 Transformando wide (largo) em long (longo) data. 64

LISTA DE TABELAS

1 Como o parâmetro *how* do **Pandas** se relaciona com o **SQL** 22
2 Comportamento do parâmetro `how` no merge 22
3 Parâmetros do `merge` 23
4 Métodos para limpeza de dados. 50
5 Comparando Normalização SQL e Tidy Data 92
6 Métodos para transformação largo (wide) e longo (long) 100

1 Objetivo

O objetivo deste estudo é explorar e documentar as funcionalidades essenciais das principais bibliotecas científicas do Python, como `NumPy`, `Pandas` e outras, através de exemplos práticos e casos de uso selecionados. Pretende-se consolidar o conhecimento sobre a manipulação, análise e visualização de dados, servindo como um guia de referência pessoal para futuros projetos de programação científica.

2 Preparação dos dados

2.1 Introdução

A essa altura, você deverá ser capaz de carregar dados no `Pandas` e fazer algumas visualização básica. Essa parte do livro tem como foco várias tarefas de limpeza dos dados. Começaremos com a preparação de um conjunto de dados para análise por meio da combinação de diversos conjuntos.

Mapa conceitual

1. Conhecimento prévio
   a) Carga de dados;
   b) Obtenção de subconjuntos de dados;
   c) Funções e métodos de classe.

Objetivos

Este capítulo abordará:

1. *Tidy data* (dados organizados);
2. Concatenação de dados;
3. Combinação (merge) de conjunto de dados.

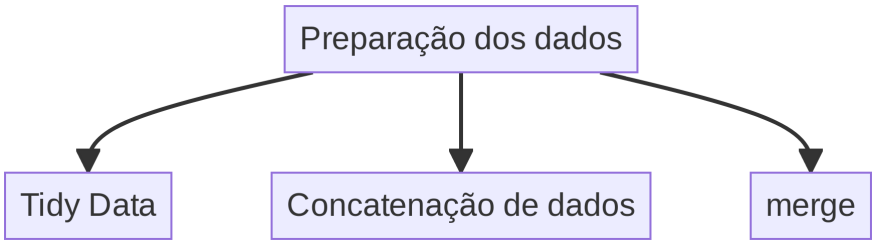

Figura 1: Preparação de dados, principais tópicos.

2.2 Tidy data

Hadley Wickham, um dos mais proeminentes membros da comunidade **R**, fala sobre a ideia de *tidy data* (dados organizados).

Com efeito, ele escreveu um artigo sobre esse conceito no *Journal of Statistical Software*. *Tidy data* é um framework para estruturar conjuntos de dados a fim de que sejam facilmente analisados. É usado principalmente como um objetivo a que devemos visar quando limpamos os dados. Depois que você compreender o que é o conceito de *tidy data*, esse conhecimento fará com que a coleta de dados seja muito mais fácil.

Então o que é *tidy data*? O artigo de Hadley Wickham o define como um conceito que atende aos seguintes critérios:

1. "**Cada observação deve formar uma linha**" (Observation)

   Uma observação é o conjunto de todas as medidas feitas em uma única unidade (ex: uma pessoa em um exame, um país em um ano específico).

   - O erro comum: Repetir a mesma observação em várias colunas ou espalhar os dados de uma mesma pessoa em tabelas diferentes sem necessidade.
   - O modo Tidy: Se você está analisando a saúde de pacientes, cada linha deve representar um paciente em um momento específico.

2. "**Cada variável deve formar uma coluna**" (Variable)

   Uma variável é um atributo que você mede (ex: Peso, Data, Temperatura).

   - O erro comum: Ter colunas chamadas "Janeiro", "Fevereiro" e "Março". Aqui, o nome da variável é "Mês", e Janeiro/Fevereiro são apenas valores.
   - O modo Tidy: Criar uma coluna única chamada Mes onde os valores são listados.

3. "**Cada tipo de unidade observacional forma uma tabela**"

   Esta regra foca na organização macro.

   Sobre o terceiro critério do tidy data, as observações devem ser coerentes com a tabela, tornando-a objetiva quanto ao tipo de informação que deve armazenar. A ideia é que a tabela tenha um propósito único; ao misturar assuntos diferentes em uma mesma estrutura, fere-se a normalização dos dados.

   - O erro comum: Misturar dados de "Clientes" com dados de "Vendas" na mesma tabela, causando redundância (ex: repetir o endereço do cliente toda vez que ele compra algo).
   - O modo Tidy: Ter uma tabela para Clientes e outra para Vendas, relacionando-as por um ID. Isso facilita a manutenção e evita erros de digitação.

2.2.1 Combinando conjuntos de dados

Começaremos com o último critério de Hadley Wickham para *tidy data*: "cada tipo de unidade de observação forma uma tabela".

Quando os dados estão organizados, é necessário combinar várias tabelas para responder a uma pergunta. Por exemplo, pode haver uma tabela separada que armazene informações de empresas e outra tabela contendo preços de ações. Se quisermos observar os preços de todas as ações no mercado de tecnologia, talvez antes tenhamos de encontrar todas as empresas de tecnologia na tabela de informações sobre empresas e então combinar esses dados com os preços das ações a fim de obter as informações de que precisamos para responder à nossa pergunta.

Os dados podem ter sido separados em tabelas distintas para reduzir a quantidade de informações redundantes (não precisamos armazenar informações sobre as empresas em cada entrada de preço das ações), mas essa organização implica que, como analistas de dados, teremos de combinar os dados relevantes por conta própria para responder à nossa pergunta.

Em outras ocasiões, um único conjunto de dados pode estar dividido em várias partes. Por exemplo, em dados de séries temporais, cada data pode estar em um arquivo separado. Em outro caso, um arquivo pode ter sido separado em partes para que os arquivos individuais fossem menores. Talvez você precise combinar dados de diversas origens para responder a uma pergunta (por exemplo, como combinar latitudes em longitudes com CEPs). Nos dois casos, você terá que combinar dados em um único dataframe de análise.

2.3 Concatenação

Uma das maneiras mais (conceitualmente) fáceis de combinar dados é por meio da concatenação.

Podemos pensar na concatenação como uma junção de linhas ou colunas em seus dados. Essa abordagem é possível se seus dados estiverem separados em partes ou se você fez um cálculo que queira concatenar ao seu conjunto de dados existente.

A concatenação é feita usando a função `concat` do `Pandas`.

2.3.1 Adicionando linhas

Vamos começar com alguns conjuntos de dados de exemplo para que você veja o que realmente acontece.

```
import pandas as pd

df1 = pd.read_csv("Cap_04-Preparacao_dados/01-Concatenacao/concat_1.csv")
df2 = pd.read_csv("Cap_04-Preparacao_dados/01-Concatenacao/concat_2.csv")
df3 = pd.read_csv("Cap_04-Preparacao_dados/01-Concatenacao/concat_3.csv")

print("dataframe_csv_1:")
print(df1)
print("dataframe_csv_2:")
print(df2)
print("dataframe_csv_3:")
print(df3)
```

```
dataframe_csv_1:
    A   B   C   D
0  a0  b0  c0  d0
1  a1  b1  c1  d1
2  a2  b2  c2  d2
3  a3  b3  c3  d3
dataframe_csv_2:
    A   B   C   D
0  a4  b4  c4  d4
1  a5  b5  c5  d5
2  a6  b6  c6  d6
3  a7  b7  c7  d7
dataframe_csv_3:
```

```
     A    B    C    D
0   a8   b8   c8   d8
1   a9   b9   c9   d9
2  a10  b10  c10  d10
3  a11  b11  c11  d11
```

A operação de empilhar dataframes uns sobre os outros é feita com a função `concat` do `Pandas`. Todos os dataframes a serem concatenados são passados em uma `list`.

```
row_concat = pd.concat([df1,df2,df3])
print(row_concat)
```

```
     A    B    C    D
0   a0   b0   c0   d0
1   a1   b1   c1   d1
2   a2   b2   c2   d2
3   a3   b3   c3   d3
0   a4   b4   c4   d4
1   a5   b5   c5   d5
2   a6   b6   c6   d6
3   a7   b7   c7   d7
0   a8   b8   c8   d8
1   a9   b9   c9   d9
2  a10  b10  c10  d10
3  a11  b11  c11  d11
```

Como podemos ver, `concat` empilha cegamente os dataframes. Se observar os nomes das linhas (isto é, seus índices), verá que eles são apenas uma versão empilhada dos índices originais das linhas.

Se aplicarmos os diversos métodos para obtenção de subconjuntos, os subconjuntos serão obtidos conforme esperado.

```
#Obtém o subconjunto da quarta linha do dataframe concatenado
print(row_concat.iloc[3,])
```

```
A    a3
B    b3
C    c3
D    d3
Name: 3, dtype: object
```

O que acontece se você usar `loc` para obter o subconjunto do novo dataframe?

A função `loc` pega os subconjuntos pelo rótulo (nome da linha), logo pega todos os rótulos com mesmo nome, pegando assim 3 linhas diferentes com mesmo nome.

```
print(row_concat.loc[3])
```

```
     A    B    C    D
3   a3   b3   c3   d3
3   a7   b7   c7   d7
3  a11  b11  c11  d11
```

2.3.1.1 Concatenar `Series`

Anteriormente apresentamos o processo de criar uma `Series`. No entanto, se criássemos uma nova série para concatenar em um dataframe, ela não seria concatenada corretamente.

```
#Criar uma nova linha de dados
new_row_series = pd.Series(['n1','n2','n3','n4'])
print(new_row_series)
```

```
0    n1
1    n2
2    n3
3    n4
dtype: object
```

```
#Tentando adicionar a nova linha em um dataframe
print(pd.concat([df1,new_row_series]))
```

```
     A    B    C    D    0
0   a0   b0   c0   d0  NaN
1   a1   b1   c1   d1  NaN
2   a2   b2   c2   d2  NaN
3   a3   b3   c3   d3  NaN
0  NaN  NaN  NaN  NaN   n1
1  NaN  NaN  NaN  NaN   n2
2  NaN  NaN  NaN  NaN   n3
3  NaN  NaN  NaN  NaN   n4
```

O primeiro detalhe que você perceba são os valores `NaN`. É simplesmente o modo Python de representar um "valor ausente".

Esperávamos concatenar nossos novos valores como uma linha, mas isso não aconteceu. De fato, nosso código não só não concatenou os valores como uma linha, como também criou uma nova coluna totalmente desalinhada em relação ao restante dos dados.

Se pararmos para pensar no que está acontecendo nesse caso, poderemos ver que o resultado, na verdade, faz sentido. Em primeiro lugar, se observarmos os novos índices adicionados, percebemos que são muito semelhantes aos resultados que obtivemos quando concatenamos dataframes antes. Os índices do objeto `new_row_series` são análogos aos números das linhas do dataframe.

Além disso, como nossa série não tem uma coluna correspondente, nosso `new_row_series` foi adicionado em uma nova coluna.

Para corrigir esse problema, podemos transformar a nossa série em um dataframe. Esse dataframe contém uma linha de dados, e os nomes das colunas são aqueles com as quais os dados serão associados.

```
#Observe os colchetes duplos
new_row_df = pd.DataFrame([['n1','n2','n3','n4']],columns=['A','B','C','D'])
print(new_row_df)
```

```
    A   B   C   D
0  n1  n2  n3  n4
```

```
print(pd.concat([df1,new_row_df]))
```

```
    A   B   C   D
0  a0  b0  c0  d0
1  a1  b1  c1  d1
2  a2  b2  c2  d2
3  a3  b3  c3  d3
0  n1  n2  n3  n4
```

`concat` é uma função genérica capaz de concatenar vários dados de uma só vez.

Se você tiver de concatenar um único objeto a um dataframe existente, a função `append` poderá cauidar dessa tarefa.

O método `.append()` foi descontinuado nas versões mais recentes do Pandas (acima da 2.0). Atualmente, a forma correta e recomendada de juntar DataFrames é usar o pd.concat().

- Usando um `DataFrame`:

```
print(df1.append(df2))
```

- Usando um `DataFrame` com uma só linha:

```
print(df1.append(new_row_df))
```

- Usando um dicionário Python:

```
data_dict = {
   'A': 'n1',
   'B': 'n2',
   'C': 'n3',
   'D': 'n4'
}

print(df1.append(data_dict, ignore_index=True))
```

2.3.1.2 Ignorando o índice

No último exemplo, quando adicionamos um `dict` em um dataframe, tivemos que usar o parâmetro `ignore_index`. Se observarmos com mais atenção, veremos que o índice da linha também foi incrementado em 1, e não houve repetição de um valor de índice anterior.

Se simplesmente queremos concatenar os dados, podemos usar o parâmetro `ignore_index` para reiniciar o índice da linha após a concatenação.

```
row_concat_i = pd.concat([df1,df2,df3],
ignore_index=True)
print(row_concat_i)
```

```
      A    B    C    D
0    a0   b0   c0   d0
1    a1   b1   c1   d1
2    a2   b2   c2   d2
3    a3   b3   c3   d3
4    a4   b4   c4   d4
5    a5   b5   c5   d5
6    a6   b6   c6   d6
7    a7   b7   c7   d7
8    a8   b8   c8   d8
9    a9   b9   c9   d9
10  a10  b10  c10  d10
11  a11  b11  c11  d11
```

2.3.2 Adicionando colunas

Concatenando colunas - axis

Concatenar colunas é muito semelhante a concatenar linhas. A principal diferença esta no parâmetro `axis` da função `concat`.

O valor default de `axis` é `0`, portanto ele concatenará os dados nas linhas. Entretanto, se passarmos `axis=1` para a função, os dados serão concatenados nas colunas.

```
col_concat = pd.concat([df1,df2,df3],
axis=1)
print(col_concat)
```

```
    A   B   C   D   A   B   C   D    A    B    C    D
0  a0  b0  c0  d0  a4  b4  c4  d4   a8   b8   c8   d8
1  a1  b1  c1  d1  a5  b5  c5  d5   a9   b9   c9   d9
2  a2  b2  c2  d2  a6  b6  c6  d6  a10  b10  c10  d10
3  a3  b3  c3  d3  a7  b7  c7  d7  a11  b11  c11  d11
```

Subconjuntos colunas

Se tentarmos obter um subconjunto de dados com base nos nomes das colunas, teremos um resultado similar aquele obtido se concatenássemos por linha e gerássemos um subconjunto pelo índice da linhas.

```
print(col_concat['A'])
```

```
    A   A    A
0  a0  a4   a8
1  a1  a5   a9
2  a2  a6  a10
3  a3  a7  a11
```

Adicionar uma única coluna

Adicionar uma única coluna em um dataframe pode ser feito diretamente, sem usar nenhuma função específica do Pandas.

Basta específicar um novo nome de coluna e o vetor que você quer que seja atribuído a essa nova coluna.

```
col_concat['new_col_list'] = ['n1','n2','n3','n4']
print(col_concat)
```

```
    A   B   C   D   A   B   C   D    A    B    C    D new_col_list
0  a0  b0  c0  d0  a4  b4  c4  d4   a8   b8   c8   d8           n1
1  a1  b1  c1  d1  a5  b5  c5  d5   a9   b9   c9   d9           n2
2  a2  b2  c2  d2  a6  b6  c6  d6  a10  b10  c10  d10           n3
3  a3  b3  c3  d3  a7  b7  c7  d7  a11  b11  c11  d11           n4
```

```
col_concat['new_col_series'] = pd.Series(['n1','n2','n3','n4'])
print(col_concat)
```

```
    A   B   C   D   A   B   C   D    A    B    C    D new_col_list  \
0  a0  b0  c0  d0  a4  b4  c4  d4   a8   b8   c8   d8           n1
1  a1  b1  c1  d1  a5  b5  c5  d5   a9   b9   c9   d9           n2
2  a2  b2  c2  d2  a6  b6  c6  d6  a10  b10  c10  d10           n3
3  a3  b3  c3  d3  a7  b7  c7  d7  a11  b11  c11  d11           n4

  new_col_series
0             n1
1             n2
2             n3
3             n4
```

Usar a função `concat` continua funcionando, desde que você lhe passe um dataframe. Essa abordagem exige um pouco mais de código desnecessário.

Reiniciando índices colunas

Por fim, podemos reiniciar os índices das colunas para que não tenhamos nomes duplicados.

```
print(pd.concat([df1,df2,df3], axis=1, ignore_index=True))
```

```
   0   1   2   3   4   5   6   7    8    9    10   11
0  a0  b0  c0  d0  a4  b4  c4  d4   a8   b8   c8   d8
1  a1  b1  c1  d1  a5  b5  c5  d5   a9   b9   c9   d9
2  a2  b2  c2  d2  a6  b6  c6  d6  a10  b10  c10  d10
3  a3  b3  c3  d3  a7  b7  c7  d7  a11  b11  c11  d11
```

2.3.3 Concatenação com índices diferentes

Os exemplos apresentados até agora partiram de pressuposto de que estávamos executando uma concatenação simples de linha ou de coluna. Também foi suposto que a(s) nova(s) linha(s) tinha(m) os mesmos nomes de colunas ou que a(s) coluna(s) tinha(m) os mesmos índices de linha.

Esta seção aborda o que acontece quando os índices das linhas e das colunas não estão alinhados.

2.3.3.1 Concatenando linhas com colunas diferentes

Vamos modificar nossos dataframes para os próximos exemplos:

```
df1.columns = ['A','B','C','D']
df2.columns = ['E','F','G','H']
df3.columns = ['A','C','F','H']
```

```
print(df1)
```

```
    A   B   C   D
0  a0  b0  c0  d0
1  a1  b1  c1  d1
2  a2  b2  c2  d2
3  a3  b3  c3  d3
```

```
print(df2)
```

```
    E   F   G   H
0  a4  b4  c4  d4
1  a5  b5  c5  d5
2  a6  b6  c6  d6
3  a7  b7  c7  d7
```

```
print(df3)
```

```
     A    C    F    H
0   a8   b8   c8   d8
1   a9   b9   c9   d9
2  a10  b10  c10  d10
3  a11  b11  c11  d11
```

Se tentarmos concatenar esses dataframes como fizemos anteriormente, os dataframes agora serão muito mais do que simplesmente empilhados uns sobre os outros. As colunas se alinharão e `NaN` preencherá qualquer área que estaja faltando.

```
row_concat = pd.concat([df1,df2,df3])
print(row_concat)
```

```
     A    B    C    D    E    F    G    H
0   a0   b0   c0   d0  NaN  NaN  NaN  NaN
1   a1   b1   c1   d1  NaN  NaN  NaN  NaN
2   a2   b2   c2   d2  NaN  NaN  NaN  NaN
3   a3   b3   c3   d3  NaN  NaN  NaN  NaN
0  NaN  NaN  NaN  NaN   a4   b4   c4   d4
1  NaN  NaN  NaN  NaN   a5   b5   c5   d5
2  NaN  NaN  NaN  NaN   a6   b6   c6   d6
3  NaN  NaN  NaN  NaN   a7   b7   c7   d7
0   a8  NaN   b8  NaN  NaN   c8  NaN   d8
1   a9  NaN   b9  NaN  NaN   c9  NaN   d9
2  a10  NaN  b10  NaN  NaN  c10  NaN  d10
3  a11  NaN  b11  NaN  NaN  c11  NaN  d11
```

Uma maneira de evitar a inclusão de valores `NaN` é manter somente as colunas que sejam compartilhadas pela lista de objetos a serem concatenados. Um parâmetro chamado `join` faz isso.

Por padrão, seu valor é `outer`, o que significa que todas as colunas serão mantidas.

Porém, [podemos definir `join='inner'` para manter somente as colunas que sejam compartilhadas entre os conjuntos de dados]{underline}.

Se tentarmos manter apenas as colunas de todos os três dataframes, teremos um dataframe vazio, pois não há nenhuma coluna em comum.

```
print(pd.concat([df1,df2,df3], join='inner'))
```

```
Empty DataFrame
Columns: []
Index: [0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3]
```

Se usarmos os dataframes que tenham colunas em comum, somente aquelas que sejam compartilhadas por todos serão devolvidas.

```
print(pd.concat([df1,df3],ignore_index=False, join='inner'))
```

```
     A    C
0   a0   c0
1   a1   c1
2   a2   c2
3   a3   c3
0   a8   b8
1   a9   b9
2  a10  b10
3  a11  b11
```

2.3.3.2 Concatenando colunas com linhas diferentes

Vamos pegar nossos dataframes e modificá-los novamente de modo que tenham índices de linha diferente. Nesse caso, estamos tomando como base as mesmas modificações de dataframe feitas anteriormente.

```
df1.index = [0,1,2,3]
df2.index = [4,5,6,7]
df3.index = [0,2,5,7]
```

```
print(df1)
```

```
    A   B   C   D
0  a0  b0  c0  d0
1  a1  b1  c1  d1
2  a2  b2  c2  d2
3  a3  b3  c3  d3
```

```
print(df2)
```

```
    E   F   G   H
4  a4  b4  c4  d4
5  a5  b5  c5  d5
6  a6  b6  c6  d6
7  a7  b7  c7  d7
```

```
print(df3)
```

```
     A    C    F    H
0   a8   b8   c8   d8
2   a9   b9   c9   d9
5  a10  b10  c10  d10
7  a11  b11  c11  d11
```

Quando concatenamos ao longo de `axis=1`, temos o mesmo resultado de concatenar ao longo de `axis=0`. Os novos dataframes serão somados por coluna, havendo correspondência entre seus respectivos índices de linha. Indicadores de valores ausentes aparecerão nas áreas em que os índices não se alinham.

```
col_concat = pd.concat([df1,df2,df3], axis=1)
print(col_concat)
```

```
     A    B    C    D    E    F    G    H    A    C    F    H
0   a0   b0   c0   d0  NaN  NaN  NaN  NaN   a8   b8   c8   d8
1   a1   b1   c1   d1  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
2   a2   b2   c2   d2  NaN  NaN  NaN  NaN   a9   b9   c9   d9
3   a3   b3   c3   d3  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
4  NaN  NaN  NaN  NaN   a4   b4   c4   d4  NaN  NaN  NaN  NaN
5  NaN  NaN  NaN  NaN   a5   b5   c5   d5  a10  b10  c10  d10
6  NaN  NaN  NaN  NaN   a6   b6   c6   d6  NaN  NaN  NaN  NaN
7  NaN  NaN  NaN  NaN   a7   b7   c7   d7  a11  b11  c11  d11
```

Assim como fizemos quando concatenamos por linha, podemos optar por manter o resultado somente quando houver índices correspondentes usando `join=inner`.

```
print(pd.concat([df1,df3],axis=1,join='inner'))
```

```
    A   B   C   D   A   C   F   H
0  a0  b0  c0  d0  a8  b8  c8  d8
2  a2  b2  c2  d2  a9  b9  c9  d9
```

2.4 Combinando vários conjuntos de dados

A seção anterior fez alusão a alguns conceitos de banco de dados. Os parâmetros `join='inner'` e o default `join='outer'` têm origem no modo de trabalhar com banco de dados quando queremos combinar tabelas.

Em vez de simplesmente ter um índice de linha ou de coluna que queremos usar para concaternar valores, às vezes podemos ter dois ou mais dataframes que queremos combinar com base em valores de dados comuns. Essa tarefa é conhecida no mundo dos bancos de dados como a execução de uma "junção" (`join`).

O Pandas tem um comando `pd.join` que usa `pd.merge` internamente. `join` fará a combinação (`merge`) de objetos com base em um índice, mas o comando `merge` é muito mais explícito e flexível. Se você planeja combinar dataframes pelo índice da linha, por exemplo, poderá consultar a função `join`.

Usaremos conjuntos de dados de pesquisa nessa série de exemplos.

```
person = pd.read_csv('./Data/Cap_04/survey_person.csv')
site = pd.read_csv('./Data/Cap_04/survey_site.csv')
survey = pd.read_csv('./Data/Cap_04/survey_survey.csv')
visited = pd.read_csv('./Data/Cap_04/survey_visited.csv')
```

```
print(person)
```

```
      ident   personal    family
0      dyer    William      Dyer
1        pb      Frank   Pabodie
2      lake   Anderson      Lake
3       roe  Valentina   Roerich
4  danforth      Frank  Danforth
```

```
print(site)
```

```
    name    lat    long
0   DR-1 -49.85 -128.57
1   DR-3 -47.15 -126.72
2  MSK-4 -48.87 -123.40
```

```r
print(survey)
```

```
    taken person quant  reading
0     619   dyer   rad     9.82
1     619   dyer   sal     0.13
2     622   dyer   rad     7.80
3     622   dyer   sal     0.09
4     734     pb   rad     8.41
5     734   lake   sal     0.05
6     734     pb  temp   -21.50
7     735     pb   rad     7.22
8     735    NaN   sal     0.06
9     735    NaN  temp   -26.00
10    751     pb   rad     4.35
11    751     pb  temp   -18.50
12    751   lake   sal     0.10
13    752   lake   rad     2.19
14    752   lake   sal     0.09
15    752   lake  temp   -16.00
16    752    roe   sal    41.60
17    837   lake   rad     1.46
18    837   lake   sal     0.21
19    837    roe   sal    22.50
20    844    roe   rad    11.25
```

```r
print(visited)
```

```
   ident   site       dated
0    619   DR-1  1927-02-08
1    622   DR-1  1927-02-10
2    734   DR-3  1939-01-07
3    735   DR-3  1930-01-12
4    751   DR-3  1930-02-26
5    752   DR-3         NaN
6    837  MSK-4  1932-01-14
7    844   DR-1  1932-03-22
```

No momento, nossos dados estão separados em várias partes, cada uma sendo uma unidade de observação. Se quiséssemos observar as datas de cada local, junto com as informações de latitude e longitude desse local, teríamos de combinar (e fazer um `merge`) de vários dataframes.Isso pode ser feito com a função `merge` do Pandas. `merge` é, na verdade, um método de `dataframe`.

Quando chamamos esse método, o dataframe chamado será referenciado como '`left`'. Na função `merge`, o primeiro parâmetro é o dataframe '`right`'. O próximo parâmetro indica como o resultado final combinado se parecerá (`how`). A Tabela 1 apresenta mais detalhes.

Em seguida, definimos o parâmetro `on`. Ele especifica com quais colunas será feita a correspondência. Se as colunas a esquerda e à direita não tiverem o mesmo nome, poderemos usar os parâmetros `left_on` e `right_on` em seu lugar.

Tabela 1: Como o parâmetro *how* do **Pandas** se relaciona com o **SQL**

Pandas	**SQL**	Descrição
`left`	`left outer`	Mantém todas as **chaves da esquerda**.
`right`	`right outer`	Mantém todas as **chaves da direita**.
`outer`	`full outer`	Mantém todas as **chaves** tanto da **esquerda** quanto da **direita**.
`inner`	`inner`	Mantém **somente** as **chaves** que existem **tanto na esquerda quanto na direita**.

Tabela 2: Comportamento do parâmetro `how` no merge

Tipo de Join	Conceito	Resultado Prático
`inner` (padrão)	Interseção	Retorna apenas as linhas que possuem chaves em ambos os DataFrames.
`left`	Prioriza a Esquerda	Mantém todos os dados do DataFrame da esquerda e traz o que houver match da direita.
`right`	Prioriza a Direita	Mantém todos os dados do DataFrame da direita e traz o que houver match da esquerda.
`outer`	União	Retorna todos os registros de ambos. Onde não houver correspondência, preenche com `NaN`.

Tabela 3: Parâmetros do `merge`

Parâmetro	Função
`left`	O primeiro DataFrame (base principal no caso de `left join`).
`right`	O segundo DataFrame (o que será "anexado").
`how`	O tipo de união: '`left`', '`right`', '`inner`' (padrão) ou '`outer`'.
`on`	Nome da coluna usada como chave (quando o nome é igual em ambos).
`left_on`	Nome da coluna chave no DataFrame da **esquerda**.
`right_on`	Nome da coluna chave no DataFrame da **direita**.
`suffixes`	Sufixos para diferenciar colunas com nomes iguais que não são chaves (ex: `_x`, `_y`).

2.4.1 `merge` um a um (one-to-one)

No tipo mais simples de `merge`, temos dois dataframes em que queremos fazer a junção de uma coluna com outra, e as colunas que queremos juntar não contém nenhum valor duplicado.

Nesse exemplo, modificamos o dataframe `visited` de modo que não haja valores duplicados de `site`.

```
visited_subset = visited.loc[[0,2,6]]
print(visited_subset)
```

```
   ident   site       dated
0    619   DR-1  1927-02-08
2    734   DR-3  1939-01-07
6    837  MSK-4  1932-01-14
```

Podemos fazer nosso `merge` um a um (***one-to-one***):

```
#O valor default de 'how' é 'inner',
# portanto, não precisa ser especificado.
o2o_merge = site.merge(
   visited_subset,
   left_on='name',
   right_on='site'
)

print(o2o_merge)
```

```
    name    lat    long  ident   site       dated
0   DR-1 -49.85 -128.57    619   DR-1  1927-02-08
1   DR-3 -47.15 -126.72    734   DR-3  1939-01-07
2  MSK-4 -48.87 -123.40    837  MSK-4  1932-01-14
```

Como pode ver, criamos agora um novo dataframe a partir de dois dataframes diferentes, em que houve uma correspondência de linhas com base em um conjunto particular de colunas. No jargão de **SQL**, as colunas usadas na correspondência são chamadas de "**chaves**".

2.4.2 `merge` de muitos para um (many-to-one)

Se optarmos por fazer o mesmo `merge`, mas dessa vez sem usar o dataframe `visited` como subconjunto, fariamos um `merge` de muitos para um (***many-to-one***). Nesse tipo de merge, um dos dataframes tem valores de chave que se repetem.

Os dataframes contendo as observações únicas serão então duplicados no `merge`.

```
m2o_merge = site.merge(
   visited,
   left_on='name',
   right_on='site'
)

print(m2o_merge)
```

```
    name    lat    long  ident   site       dated
0   DR-1 -49.85 -128.57    619   DR-1  1927-02-08
1   DR-1 -49.85 -128.57    622   DR-1  1927-02-10
2   DR-1 -49.85 -128.57    844   DR-1  1932-03-22
3   DR-3 -47.15 -126.72    734   DR-3  1939-01-07
4   DR-3 -47.15 -126.72    735   DR-3  1930-01-12
5   DR-3 -47.15 -126.72    751   DR-3  1930-02-26
6   DR-3 -47.15 -126.72    752   DR-3         NaN
7  MSK-4 -48.87 -123.40    837  MSK-4  1932-01-14
```

Como podemos ver, as informações de `site`(`name`, `lat` e `long`) foram duplicadas e houve correspondência com os dados de `visited`.

2.4.3 `merge` de muitos para muitos (many-to-many)

Por fim, há ocasiões em que vamos querer efetuar uma correspondência baseada em várias colunas. Como exemplo, suponha que haja dois dataframes, provenientes de um `merge` entre `person` e `survey` e de outro entre `visited` com `survey`.

```
ps = person.merge(
   survey,
   left_on='ident',
   right_on='person'
)

vs = visited.merge(
   survey,
   left_on='ident',
   right_on='taken'
)
```

```
print(ps)
```

```
   ident   personal   family  taken person quant  reading
0   dyer    William     Dyer    619   dyer   rad     9.82
1   dyer    William     Dyer    619   dyer   sal     0.13
2   dyer    William     Dyer    622   dyer   rad     7.80
3   dyer    William     Dyer    622   dyer   sal     0.09
4     pb      Frank  Pabodie    734     pb   rad     8.41
5     pb      Frank  Pabodie    734     pb  temp   -21.50
6     pb      Frank  Pabodie    735     pb   rad     7.22
7     pb      Frank  Pabodie    751     pb   rad     4.35
8     pb      Frank  Pabodie    751     pb  temp   -18.50
9   lake   Anderson     Lake    734   lake   sal     0.05
10  lake   Anderson     Lake    751   lake   sal     0.10
11  lake   Anderson     Lake    752   lake   rad     2.19
12  lake   Anderson     Lake    752   lake   sal     0.09
13  lake   Anderson     Lake    752   lake  temp   -16.00
14  lake   Anderson     Lake    837   lake   rad     1.46
15  lake   Anderson     Lake    837   lake   sal     0.21
16   roe  Valentina  Roerich    752    roe   sal    41.60
17   roe  Valentina  Roerich    837    roe   sal    22.50
18   roe  Valentina  Roerich    844    roe   rad    11.25
```

```r
print(vs)
```

```
    ident   site       dated  taken person quant  reading
0     619   DR-1  1927-02-08    619   dyer   rad     9.82
1     619   DR-1  1927-02-08    619   dyer   sal     0.13
2     622   DR-1  1927-02-10    622   dyer   rad     7.80
3     622   DR-1  1927-02-10    622   dyer   sal     0.09
4     734   DR-3  1939-01-07    734     pb   rad     8.41
5     734   DR-3  1939-01-07    734   lake   sal     0.05
6     734   DR-3  1939-01-07    734     pb  temp   -21.50
7     735   DR-3  1930-01-12    735     pb   rad     7.22
8     735   DR-3  1930-01-12    735    NaN   sal     0.06
9     735   DR-3  1930-01-12    735    NaN  temp   -26.00
10    751   DR-3  1930-02-26    751     pb   rad     4.35
11    751   DR-3  1930-02-26    751     pb  temp   -18.50
12    751   DR-3  1930-02-26    751   lake   sal     0.10
13    752   DR-3         NaN    752   lake   rad     2.19
14    752   DR-3         NaN    752   lake   sal     0.09
15    752   DR-3         NaN    752   lake  temp   -16.00
16    752   DR-3         NaN    752    roe   sal    41.60
17    837  MSK-4  1932-01-14    837   lake   rad     1.46
18    837  MSK-4  1932-01-14    837   lake   sal     0.21
19    837  MSK-4  1932-01-14    837    roe   sal    22.50
20    844   DR-1  1932-03-22    844    roe   rad    11.25
```

Podemos executar um merge de muitos para muitos (***many-to-many***) passando as várias colunas com as quais a correspondência será feita, usando uma lista Python.

```
ps_vs = ps.merge(
   vs,
   left_on=['ident','taken','quant','reading'],
   right_on=['person','ident','quant','reading']
)
```

Vamos observar apenas a primeira linha:

```
print(ps_vs.loc[0,])
```

```
ident_x           dyer
personal       William
family            Dyer
taken_x            619
person_x          dyer
quant              rad
reading           9.82
ident_y            619
site              DR-1
dated       1927-02-08
taken_y            619
person_y          dyer
Name: 0, dtype: object
```

O Pandas acrescentará automaticamente um sufixo no nome de uma coluna se houver colisões no nome. Na saída, `_x` refere-se aos valores do dataframe à esquerda, enquanto o sufixo `_y` é proveniente dos valores do dataframe à direita.

2.4.4 Resumo

O `pd.merge()` funciona de forma similar aos `JOIN`s do **SQL**. O que define o tipo de relação é a frequência com que as chaves de ligação aparecem em cada DataFrame.

1. ***One-to-One*** (Um-para-Um - o2o)

   Ocorre quando a chave de ligação é única em ambos os DataFrames. É como unir duas tabelas de informações diferentes sobre os mesmos indivíduos.

   - **A lógica**: Cada linha da Tabela A encontra exatamente uma linha correspondente na Tabela B.
   - **Impacto nas linhas**: O número de linhas do DataFrame resultante será igual ao número de chaves que coincidem. Nenhuma linha é duplicada; as colunas são apenas "esticadas" para o lado.

   Exemplo: Uma tabela com `ID` e `Nome`, e outra com `ID` e `Cargo`.

2. ***Many-to-One*** (Muitos-para-Um - m2o)

   Ocorre quando uma das tabelas possui chaves repetidas, mas a outra possui apenas chaves únicas. O pandas preserva as repetições e propaga os dados da tabela "única".

   - **A lógica**: Uma linha da Tabela B (o lado "One") corresponde a várias linhas da Tabela A (o lado "Many").
   - **Impacto nas linhas**: As linhas da Tabela A são preservadas. O conteúdo da linha correspondente na Tabela B é copiado para cada uma dessas linhas.
   - **Exemplo visual**: Se o "Vendedor 1" aparece em 10 linhas de vendas, os dados dele (nome, região) serão repetidos nessas 10 linhas no resultado final.

   Exemplo: Uma tabela de **Vendas** (onde o `ID_Vendedor` se repete) e uma tabela de **Vendedores** (onde cada `ID_Vendedor` é único).

3. ***Many-to-Many*** (Muitos-para-Muitos - m2m)

   Ocorre quando a chave de ligação se repete em ambos os DataFrames. O resultado é um produto cartesiano das linhas correspondentes.

   - **A lógica**: Uma chave aparece N vezes na Tabela A e M vezes na Tabela B.
   - **Impacto nas linhas**: O Pandas cria todas as combinações possíveis entre elas. O número de linhas resultantes para aquela chave específica será $N \times M$.
   - **Atenção**: Este é o caso mais perigoso, pois pode gerar um aumento explosivo no tamanho do seu DataFrame se você não estiver esperando por essas combinações.

Exemplo: Uma tabela de Produtos em promoção e uma tabela de Lojas que vendem esses produtos. Se um produto aparece 3 vezes na Tabela A e 2 vezes na Tabela B, o resultado terá $3 \times 2 = 6$ linhas para essa chave.

2.5 Conclusão preparação dos dados

As vezes, será preciso combinar diversas partes ou dados ou vários conjuntos de dados, conforme a pergunta que tiver tentano responder. Tenha em mente, porém, que os dados de que você precisar nas análises não estarão necessariamente no melhor formato usado para armazená-los.

Os dados de pesquisa usados no último exemplo estavam separados em quatro partes que precisam ser combinadas. Depois que fizemos o `merge` das tabelas, muitas informações redundantes apareceram nas linhas. Do ponto de vista da armazenagem de dados e de sua entrada, cada uma dessas duplicações pode lever a erros e inconsistência nos dados. É isso que Hadley quis dizer quando afirmou que, nos dados organizados (***Tidy data***), "cada tipo de unidade de observação forma uma tabela".

3 Dados Ausentes

3.1 Introdução

Raramente você receberá um conjunto de dados sem valor ausente. Há muitas representações para dados ausentes. Nos bancos de dados, são valores `NULL`; certas linguagens de programação usam `NA`, e, dependendo da origem de seus dados, valores ausentes podem ser uma string vazia, ' ', ou até mesmo valores númericos como `88` ou `99`. O Pandas exibe valores ausentes como `NaN`.

Mapa conceitual

1. Conhecimento prévio:

    a. Importação de bibliotecas;
    b. Fatiamento e indexação de dados;
    c. Uso de funções e métodos;
    d. Uso de parâmetros de funções.

Objetivos

Este capítulo abordará:

1. O que é um valor ausente
2. Como os valores ausentes são criados
3. Como recodificar e fazer cálculos com valores ausentes

3.2 O que é um valor `NaN`?

O valor `NaN` no `Pandas` é proveniente do `numpy`. Valores ausentes podem ser usados ou exibidos de algumas maneiras em Python - `NaN`, `NAN` ou `nan` -, mas são todos equivalentes.

```
# Basta importar os valores ausentes do numpy

from numpy import NaN, NAN, nan
```

Valores ausentes são diferentes de outros tipos de dados, pois não são realmente iguais a nada. O dado está ausente, portanto não há um conceito de igualdade. `NaN` não é equivalente a 0 nem a uma string vazia, ''.

Podemos demonstrar essa ideia em Python fazendo testes de igualdade:

```
print(np.nan == True)
```

```
False
```

```
print(np.nan == False)
```

```
False
```

```
print(np.nan == 0)
```

```
False
```

```
print(np.nan == '')
```

```
False
```

Valores ausentes também não são iguais a outros valores ausentes:

```
print(np.nan == np.nan)
```

```
False
```

O Pandas tem métodos embutidos para testar um valor ausente:

```
print(pd.isnull(np.nan))
```

```
True
```

O Pandas também tem métodos para testar valores que não são ausentes:

```
print(pd.notnull(np.nan))
```

```
False
```

```
print(pd.notnull(42))
```

```
True
```

```
print(pd.notnull('missing'))
```

```
True
```

3.3 De onde vêm os valores ausentes?

Podemos obter valores ausentes quando carregamos um conjunto de dados com esses valores, ou do processo de manipulação dos dados (***data munging***).

3.3.1 Carga de dados

Os dados de pesquisa que usamos anteriormente incluíam um conjunto de dados, `visited`, que continham dados ausentes. Quando os valores foram carregados, o Pandas encontrou automaticamente as células com dados ausentes e nos deu um dataframe com o valor `NaN` nas células apropriadas.

Na função `read_csv`, três parâmetros estão relacionados com a leitura de valores ausentes:

- `na_values`;
- `keep_default_na`;
- `na_filter`.

na_values

O parâmetro `na_values` nos permite especificar valores ausentes ou `NaN` adicionais. Você pode passar uma str Python ou um objeto do tipo lista para que seja automaticamente codificados como valores ausentes quando o arquivo é lido.

É claro que valores ausentes default, como `NA`, `NaN`, ou `nan`, já estão disponíveis, e é por isso que esse parâmetro nem sempre é usado.

Alguns dados saudáveis podem codificar `99` como valor ausente; para especificar o uso desse valor, defina `na_values=[99]`.

`na_values`: Permite que você adicione seus próprios marcadores (ex: '999', 'FALTA', 'INTERROGAÇÃO').

```
# Define o local em que estão os dados
visited_file = 'Data/Cap_04/survey_visited.csv'

# Carragar os dados com valores default
print(pd.read_csv(visited_file))
```

```
   ident   site       dated
0    619   DR-1  1927-02-08
1    622   DR-1  1927-02-10
2    734   DR-3  1939-01-07
3    735   DR-3  1930-01-12
4    751   DR-3  1930-02-26
5    752   DR-3         NaN
6    837  MSK-4  1932-01-14
7    844   DR-1  1932-03-22
```

keep_default_na

O parâmetro `keep_default_na` é um `bool` que permite especificar se algum valor adicional deve ser considerado como ausente.

Esse valor é `True` por padrão, ou seja, qualquer valor ausente adicional especificado com o parâmetro `na_values` será concatenado à lista de valores ausentes. No entando, `keep_default_na` também pode ser definido com `keep_default_na=False`; nesse caso, somente os valores ausentes especificados em `na_values` serão usados.

`keep_default_na=False` decide se a lista padrão do Pandas (ex: '`NaN`', '`NULL`', '`N/A`') deve ser usada.

```
# Carrega os dados sem valores ausentes default
print(pd.read_csv(visited_file,keep_default_na=False))
```

```
   ident   site       dated
0    619   DR-1  1927-02-08
1    622   DR-1  1927-02-10
2    734   DR-3  1939-01-07
3    735   DR-3  1930-01-12
4    751   DR-3  1930-02-26
5    752   DR-3
6    837  MSK-4  1932-01-14
7    844   DR-1  1932-03-22
```

```
# Especifica manualmente os valores ausentes
print(pd.read_csv(
   visited_file,
   na_values=['''],
   keep_default_na=False))
```

```
   ident   site       dated
0    619   DR-1  1927-02-08
1    622   DR-1  1927-02-10
2    734   DR-3  1939-01-07
3    735   DR-3  1930-01-12
4    751   DR-3  1930-02-26
5    752   DR-3         NaN
6    837  MSK-4  1932-01-14
7    844   DR-1  1932-03-22
```

`na_filter`

Por fim, `na_filter` é um `bool` que especificará se algum valor será lido como ausente.

O valor default de `na_filter=True` implica que valores ausentes serão codificados como `NaN`. Se atribuirmos `na_filter=False`, então nada será recodificado como ausente. Liga e desliga a busca por dados faltantes, se for `False`, os outros dois parâmetros são ignorados..

Podemos pensar nesse parâmetro como uma forma de desativar todos os parâmetros definidos para `na_values` e `keep_default_na`, mas é mais provável que ele seja usado se quisermos ter um ganho de desempenho carregando dados sem valores ausentes.

Como ele funciona:

- `True` (Padrão): O pandas analisa o conteúdo de todas as colunas em busca de strings que representem dados faltantes e as converte para `NaN`.
- `False`: O pandas ignora essa busca. Tudo o que estiver no arquivo será lido literalmente como string ou número, sem conversão para "ausente".

Por que usar `na_filter=False`?

- Velocidade: Em arquivos gigantes onde você sabe que não há dados faltantes, desativar o filtro acelera consideravelmente o carregamento.
- Integridade: Se o seu arquivo contém a string "NA" (por exemplo, o código de um país como a Namíbia) e você não quer que o pandas a transforme em um buraco vazio (`NaN`).

3.3.2 Dados combinados

No capítulo Preparação dos dados mostrou como combinar conjunto de dados (`merge`).Alguns dos exemplos naquele capítulo inclíam valores ausentes na saída. Se recriarmos a tabela combinada, veremos valores ausentes na saída resultante.

```
visited = pd.read_csv('Data/Cap_04/survey_visited.csv')
survey = pd.read_csv('Data/Cap_04/survey_survey.csv')
```

```
print(visited)
```

```
   ident   site       dated
0    619   DR-1  1927-02-08
1    622   DR-1  1927-02-10
2    734   DR-3  1939-01-07
3    735   DR-3  1930-01-12
4    751   DR-3  1930-02-26
5    752   DR-3         NaN
6    837  MSK-4  1932-01-14
7    844   DR-1  1932-03-22
```

```
print(survey)
```

```
    taken person quant  reading
0     619   dyer   rad     9.82
1     619   dyer   sal     0.13
2     622   dyer   rad     7.80
3     622   dyer   sal     0.09
4     734     pb   rad     8.41
5     734   lake   sal     0.05
6     734     pb  temp   -21.50
7     735     pb   rad     7.22
8     735    NaN   sal     0.06
9     735    NaN  temp   -26.00
10    751     pb   rad     4.35
11    751     pb  temp   -18.50
12    751   lake   sal     0.10
13    752   lake   rad     2.19
14    752   lake   sal     0.09
15    752   lake  temp   -16.00
16    752    roe   sal    41.60
```

```
17    837   lake   rad     1.46
18    837   lake   sal     0.21
19    837    roe   sal    22.50
20    844    roe   rad    11.25
```

```python
# Default how = 'inner'
vs = visited.merge(
   survey,
   left_on='ident',
   right_on='taken'
   )
print(vs)
```

```
    ident   site       dated  taken person quant  reading
0     619   DR-1  1927-02-08    619   dyer   rad     9.82
1     619   DR-1  1927-02-08    619   dyer   sal     0.13
2     622   DR-1  1927-02-10    622   dyer   rad     7.80
3     622   DR-1  1927-02-10    622   dyer   sal     0.09
4     734   DR-3  1939-01-07    734     pb   rad     8.41
5     734   DR-3  1939-01-07    734   lake   sal     0.05
6     734   DR-3  1939-01-07    734     pb  temp   -21.50
7     735   DR-3  1930-01-12    735     pb   rad     7.22
8     735   DR-3  1930-01-12    735    NaN   sal     0.06
9     735   DR-3  1930-01-12    735    NaN  temp   -26.00
10    751   DR-3  1930-02-26    751     pb   rad     4.35
11    751   DR-3  1930-02-26    751     pb  temp   -18.50
12    751   DR-3  1930-02-26    751   lake   sal     0.10
13    752   DR-3         NaN    752   lake   rad     2.19
14    752   DR-3         NaN    752   lake   sal     0.09
15    752   DR-3         NaN    752   lake  temp   -16.00
16    752   DR-3         NaN    752    roe   sal    41.60
17    837  MSK-4  1932-01-14    837   lake   rad     1.46
18    837  MSK-4  1932-01-14    837   lake   sal     0.21
19    837  MSK-4  1932-01-14    837    roe   sal    22.50
20    844   DR-1  1932-03-22    844    roe   rad    11.25
```

3.3.3 Valores de entrada do usuário

O usuário tambem pode criar valores ausentes - por exemplo, criar um vetor de valores a partir de um cálculo ou um vetor preparado manualmente.

Criaremos nossos próprios dados com valores ausentes. `NaN`s são valores válidos tanto quanto para `Series` quanto para `DataFrames`.

```
# Valores ausentes em uma Series
# Também poderia usar 'None' ao inves de pd.nan
num_legs = pd.Series({'goat':4, 'amoeba': None})

print(num_legs)
```

```
goat      4.0
amoeba    NaN
dtype: float64
```

```
# Valores ausentes em um dataframe
scientists = pd.DataFrame({
   'Name':['Rosaline Franklin','William Gosset'],
   'Occupation':['Chemist','statistician'],
   'Born':['1920-07-25','1876-06-13'],
   'Died':['1958-04-16','1937-10-16'],
   'missing': [None, np.nan]
})

print(scientists)
```

```
                Name    Occupation        Born        Died  missing
0  Rosaline Franklin       Chemist  1920-07-25  1958-04-16      NaN
1     William Gosset  statistician  1876-06-13  1937-10-16      NaN
```

Você também pode atribuir diretamente uma coluna de valores ausentes a um dataframe.

```
# Cria um novo dataframe
scientists = pd.DataFrame({
   'Name':['Rosaline Franklin','William Gosset'],
   'Occupation':['Chemist','statistician'],
   'Born':['1920-07-25','1876-06-13'],
   'Died':['1958-04-16','1937-10-16']
})

# Atrabui uma coluna de valores ausentes
scientists['missing']=np.nan

print(scientists)
```

```
                Name    Occupation        Born        Died  missing
0  Rosaline Franklin       Chemist  1920-07-25  1958-04-16      NaN
1     William Gosset  statistician  1876-06-13  1937-10-16      NaN
```

3.3.4 Reindexação

Outra maneira de introduzir valores ausentes em seus dados é reindexando o seu dataframe. Isso é conveniente quando queremos adicionar novos indices em nosso dataframe, mas queremos preservar seus valores originais. Um uso comum é quando o índice representa algum intervalo de tempo e queremos adicionar mais datas.

Em termos simples:

- O que acontece: Você define uma nova lista de índices (ex: novas datas).
- A consequência: O Pandas mantém os dados que já existiam e, para os novos índices que não possuem valor correspondente, ele insere automaticamente um NaN (valor ausente).

Se quiséssemos observar somente os anos 2000 a 2010 da plotagem de dados Gapminder, poderíamos executar asmesmas operações agrupadas, obter um subconjunto dos dados e então os reindexar.

```
# Carregando arquivo CSV
gapminder = pd.read_csv('Data/Cap_01/gapminder.tsv', sep='\t')

# Agrupamento de dados
life_exp = gapminder.groupby(['year'])['lifeExp'].mean()

print(life_exp)
```

```
year
1952    49.057620
1957    51.507401
1962    53.609249
1967    55.678290
1972    57.647386
1977    59.570157
1982    61.533197
1987    63.212613
1992    64.160338
1997    65.014676
2002    65.694923
2007    67.007423
Name: lifeExp, dtype: float64
```

Podemos reindexar fatiando os dados.

```
# Você pode continuar a encadear o 'loc' do código anterior
# Esse código não funciona mais!
print(life_exp.loc[range(2000,2010),])
```

No Pandas moderno (versão 1.0 ou superior), se você tentar usar o `.loc` com uma lista de rótulos onde alguns não existem no índice, o código gerará um erro de KeyError.

Antigamente, o Pandas criava linhas com NaN automaticamente, mas isso mudou para evitar erros silenciosos que poderiam causar confusão.

```
# Em vez de .loc, use .reindex
indices_desejados = range(2000, 2010)
print(life_exp.\
   reindex(indices_desejados))
```

```
year
2000          NaN
2001          NaN
2002    65.694923
2003          NaN
2004          NaN
2005          NaN
2006          NaN
2007    67.007423
2008          NaN
2009          NaN
Name: lifeExp, dtype: float64
```

Como alternativa, podemos obter o subconjunto dos dados separadamente e usar o método `reindex`.

```
# Obtém subconjunto
y2000 = life_exp[life_exp.index > 2000]
print(y2000)
```

```
year
2002    65.694923
2007    67.007423
Name: lifeExp, dtype: float64
```

```
# Reindexa
print(y2000.reindex(range(2000,2010)))
```

```
year
2000          NaN
2001          NaN
2002    65.694923
2003          NaN
2004          NaN
2005          NaN
2006          NaN
2007    67.007423
2008          NaN
2009          NaN
Name: lifeExp, dtype: float64
```

3.4 Trabalhando com dados ausentes

Agora que sabemos como os dados ausentes podem ser criados, vamos ver como eles se comportam quando estamos trabalhando com dados.

3.4.1 Encontrando e contando dados ausentes

```
ebola = pd.read_csv('Data/Cap_04/country_timeseries.csv')
```

Linhas

Uma maneira de ver o número de valores ausentes é contá-los com `count`.

```
print(ebola.count())
```

```
Date                   122
Day                    122
Cases_Guinea            93
Cases_Liberia           83
Cases_SierraLeone       87
Cases_Nigeria           38
Cases_Senegal           25
Cases_UnitedStates      18
Cases_Spain             16
Cases_Mali              12
Deaths_Guinea           92
Deaths_Liberia          81
Deaths_SierraLeone      87
Deaths_Nigeria          38
Deaths_Senegal          22
Deaths_UnitedStates     18
Deaths_Spain            16
Deaths_Mali             12
dtype: int64
```

Você também pode subtrair o número de linhas não ausentes do número total de linhas.

```
num_rows = ebola.shape[0]
num_missing = num_rows - ebola.count()
print("Número de linhas ausentes:")
print(num_missing)
```

```
Número de linhas ausentes:
Date                     0
Day                      0
Cases_Guinea            29
Cases_Liberia           39
Cases_SierraLeone       35
Cases_Nigeria           84
Cases_Senegal           97
Cases_UnitedStates     104
Cases_Spain            106
Cases_Mali             110
Deaths_Guinea           30
Deaths_Liberia          41
Deaths_SierraLeone      35
Deaths_Nigeria          84
Deaths_Senegal         100
Deaths_UnitedStates    104
Deaths_Spain           106
Deaths_Mali            110
dtype: int64
```

Colunas

Se quiser contar o número total de valores ausentes em seus dados, ou contar o número de valores ausentes em uma coluna em particular, você poderá usar a função `count_nonzero` do `numpy` em conjunto com o método `isnull`.

A função `np.count_nonzero()` conta quantos valores são verdadeiros (`True`). Como o método `.isnull()` retorna `True` para cada dado ausente, a combinação dos dois resulta no total de nulos.

```
print(np.count_nonzero(ebola.isnull()))
```

```
1214
```

```
print(np.count_nonzero(ebola['Cases_Guinea'].isnull()))
```

```
29
```

Outra forma de obter os contadores de dados ausentes é usar o método `value_counts` em uma série. Uma tabela de frequência de valores será exibida. Se o parâmetro `dropna` for usado, você também poderá obter um contador de valores ausentes.

O `value_counts` serve para contar quantas vezes cada valor aparece em uma coluna. Imagine uma coluna com as frutas: [Maçã, Maçã, Banana, NaN]. O que acontece na prática:

- Sem o `dropna=False` (Comportamento padrão): O Pandas ignora os dados ausentes. Ele diria que você tem 2 Maçãs e 1 Banana. O `NaN` fica invisível.
- Com o `dropna=False`: O Pandas passa a tratar o dado ausente como uma categoria comum. O resultado seria:
  - Maçã: 2
  - Banana: 1
  - NaN: 1

```
# Obtém os 5 primeiros contadores da coluna Cases_Guinea
print(ebola.Cases_Guinea.value_counts(dropna=False).head())
```

```
Cases_Guinea
NaN      29
86.0      3
112.0     2
495.0     2
390.0     2
Name: count, dtype: int64
```

```
# Também poderia ser
print(ebola['Cases_Guinea'].value_counts(dropna=False).head())
```

```
Cases_Guinea
NaN      29
86.0      3
112.0     2
495.0     2
390.0     2
Name: count, dtype: int64
```

3.4.2 Limpando dados ausentes

Há muitas maneiras com as quais podemos lidar com dados ausentes. Por exemplo, podemos substituir os dados ausentes por outro valor, preenchê-los usando dados existentes ou descartá-los de nosso conjunto de dados.

Tabela 4: Métodos para limpeza de dados.

Método	Descrição	Exemplo
`fillna`	Recodificar/substituir valores ausentes.	`df.fillna(0)`
`method='ffill'`	Método embutido para fazer preenchimento para a frente (forward).	`.fillna(method='ffill')`
`method='bfill'`	Método embutido para fazer preenchimento para trás (backward).	`.fillna(method='bfill')`
`interpolate`	A interpolação usa valores existentes para preencher dados ausentes.	`.interpolate()`
`dropna`	Descartar observações ou variáveis com esses dados ausentes.	.dropna()

3.4.2.1 Recodificar/substituir

Podemos usar o método `fillna` para recodificar os valores ausentes, transformando-os em outro valor.

Por exemplo, suponha que quiséssemos que os valores ausentes fossem recodificados como 0.

```
print(ebola.fillna(0).iloc[0:10,0:5])
```

```
         Date  Day  Cases_Guinea  Cases_Liberia  Cases_SierraLeone
0    1/5/2015  289        2776.0            0.0            10030.0
1    1/4/2015  288        2775.0            0.0             9780.0
2    1/3/2015  287        2769.0         8166.0             9722.0
3    1/2/2015  286           0.0         8157.0                0.0
4  12/31/2014  284        2730.0         8115.0             9633.0
5  12/28/2014  281        2706.0         8018.0             9446.0
6  12/27/2014  280        2695.0            0.0             9409.0
7  12/24/2014  277        2630.0         7977.0             9203.0
8  12/21/2014  273        2597.0            0.0             9004.0
9  12/20/2014  272        2571.0         7862.0             8939.0
```

Ao usar `fillna`, podemos recodificar os valores para um valor específico.

Se consultar a documentação, você descobrirá que `fillna`, como muitas outras funções Pandas, tem um parâmetro `inplace`. Isso significa que os dados subjacentes serão automaticamente alterados para você; não é necessário criar uma nova cópia com as alterações. Você poderá usar esse parâmetro quando seus dados ficarem maiores e quiser que o código seja mais eficiente no que diz respeito à memória.

Em termos simples:

- Comportamento padrão: Quando você usa `df.fillna(0)`, o Pandas cria uma cópia inteira da sua tabela na memória com os novos valores. O original permanece intocado a menos que você o sobrescreva (`df = df.fillna(0)`).
- Com `inplace=True`: O Pandas altera diretamente a tabela original. Ele não cria uma cópia nova.

Sintaxe do `inplace`:

```
df.fillna(0, inplace=True)
```

3.4.2.2 Preenchimento para a frente (*forward*)

Podemos usar métodos embutidos para fazer preenchimentos para a frente (*forward*) ou para trás (*backward*). Quando preenchemos os dados para a frente, o último valor conhecido é usado para o próximo valor ausente. Desse modo, os valores ausentes são substituídos pelo último valor conhecido/registrado.

```
print(ebola.fillna(method='ffill').\
   iloc[0:10,0:5])
```

```
         Date  Day  Cases_Guinea  Cases_Liberia  Cases_SierraLeone
0    1/5/2015  289        2776.0            NaN            10030.0
1    1/4/2015  288        2775.0            NaN             9780.0
2    1/3/2015  287        2769.0         8166.0             9722.0
3    1/2/2015  286        2769.0         8157.0             9722.0
4  12/31/2014  284        2730.0         8115.0             9633.0
5  12/28/2014  281        2706.0         8018.0             9446.0
6  12/27/2014  280        2695.0         8018.0             9409.0
7  12/24/2014  277        2630.0         7977.0             9203.0
8  12/21/2014  273        2597.0         7977.0             9004.0
9  12/20/2014  272        2571.0         7862.0             8939.0
```

Se a coluna começar com um valor ausente, esse dado permanecerá ausente, pois não haverá um valor anterior com o qual fazer o preenchimento.

Lógica: O valor da linha n é copiado para a linha $n + 1$ se esta for `NaN`.

3.4.2.3 Preenchimento para trás (*backward*)

Também podemos fazer com que o Pandas preencha dados para trás. Quando preenchemos dados para trás, o valor mais recente é usado para substituir os dados ausentes. Desse modo, os valores ausentes são substituídos pelo valor mais recente.

```
print(ebola.fillna(method='bfill')\
   .iloc[:,0:5]\
      .tail())
```

```
          Date  Day  Cases_Guinea  Cases_Liberia  Cases_SierraLeone
117  3/27/2014    5         103.0            8.0                6.0
118  3/26/2014    4          86.0            NaN                NaN
119  3/25/2014    3          86.0            NaN                NaN
120  3/24/2014    2          86.0            NaN                NaN
121  3/22/2014    0          49.0            NaN                NaN
```

Se uma coluna terminar com um valor ausente, esse dado permanecerá ausente, pois não haverá nenhum valor novo com o qual fazer o preenchimento.

Lógica: O valor da linha n é preenchido pelo valor da linha $n + 1$.

3.4.2.4 Interpolação

A interpolação usa valores existentes para preencher valores ausentes. Embora haja muitas meneiras de preencher valores ausentes, a interpolação no Pandas preenche os valores ausentes de modo linear. Especificamente, os valores ausentes são tratados como se devessem estar igualmente espaçados.

```
print(ebola.interpolate()\
   .iloc[0:10,0:5])
```

```
         Date  Day  Cases_Guinea  Cases_Liberia  Cases_SierraLeone
0    1/5/2015  289        2776.0            NaN            10030.0
1    1/4/2015  288        2775.0            NaN             9780.0
2    1/3/2015  287        2769.0         8166.0             9722.0
3    1/2/2015  286        2749.5         8157.0             9677.5
4  12/31/2014  284        2730.0         8115.0             9633.0
5  12/28/2014  281        2706.0         8018.0             9446.0
6  12/27/2014  280        2695.0         7997.5             9409.0
7  12/24/2014  277        2630.0         7977.0             9203.0
8  12/21/2014  273        2597.0         7919.5             9004.0
9  12/20/2014  272        2571.0         7862.0             8939.0
```

O método `interpolate` tem um parâmetro `method` que pode alterar o método de interpolação.

Principais Parâmetros:

- `method`: Define a técnica matemática.
  - '`linear`': (Padrão) Ignora o índice e trata os valores como igualmente espaçados.
  - '`time`': Funciona com séries temporais, considerando o intervalo real de tempo entre as datas.
  - '`polynomial`' ou '`spline`': Requer que você passe uma order (ex: ordem 2 ou 3) para curvas mais suaves.
- `limit_direction`: Controla a direção do preenchimento.
  - '`forward`';
  - '`backward`';
  - '`both`'.
- `inplace`: Se `True`, altera o DataFrame original.

3.4.2.5 Descartando valores ausentes

O último modo de trabalhar com dados ausentes é descartar observações ou variáveis com esses dados.

Podemos usar o método `dropna` para descartar dados ausentes e especificar parâmetros para que esse método que controlem como os dados são descartados.

Por exemplo, o parâmetro `how` permite especificar se uma linha (ou coluna) será descartada quando algum dado ('`any`') ou todos ('`all`') estiverem ausentes.

O parâmetro `thresh` permite especificar quantos valores diferentes de `NaN` devemos ter para descartar a linha ou a coluna.

```
print(ebola.shape)
```

```
(122, 18)
```

Se somente os casos de dados completos forem mantidos em nosso conjunto de dados Ebola, ficaremos com apenas uma linha de dados.

```
ebola_dropna = ebola.dropna()
print(ebola_dropna.shape)
```

```
(1, 18)
```

```
print(ebola_dropna)
```

```
          Date  Day  Cases_Guinea  Cases_Liberia  Cases_SierraLeone  \
19  11/18/2014  241        2047.0         7082.0             6190.0

    Cases_Nigeria  Cases_Senegal  Cases_UnitedStates  Cases_Spain  Cases_Mali  \
19           20.0            1.0                 4.0          1.0         6.0

    Deaths_Guinea  Deaths_Liberia  Deaths_SierraLeone  Deaths_Nigeria  \
19         1214.0          2963.0              1267.0             8.0

    Deaths_Senegal  Deaths_UnitedStates  Deaths_Spain  Deaths_Mali
19             0.0                  1.0           0.0          6.0
```

Principais parâmetros:

- `axis`:

  Define se você remove a linha (`0` ou '`index`') ou a coluna (`1` ou '`columns`'). O padrão é remover a linha.

- `how`:

  - '`any`': Remove se houver pelo menos um valor nulo (**padrão**).
  - '`all`': Só remove se todos os valores da linha/coluna forem nulos.

- `subset`:

  Permite focar em colunas específicas.

  Ex: `df.dropna(subset=['idade'])` só removerá a linha se o nulo estiver na coluna "`idade`".

- `inplace=True`:

  Altera o DataFrame original diretamente, sem precisar atribuir a uma nova variável.

Cuidado:

Dependendo da quantidade de dados que estiver faltando, manter somente os casos com dados completos pode deixar você com um conjunto de dados inútil. Talvez os dados ausentes não sejam aleatórios, de modo que descartar valores ausentes fará com que você fique com um conjunto de dados tendenciosos ou, quem sabe, manter somente os dados completos o deixará com dados insuficientes para efetuar a sua análise.

3.4.3 Cálculos com dados ausentes

Supanha que quiséssemos ver os contadores de casos para várias regiões. Podemos somar vários regiões a fim de obter uma nova coluna que armazene os contadores de casos.

```
ebola['Cases_multiple'] = ebola['Cases_Guinea']+\
   ebola['Cases_Liberia']+\
   ebola['Cases_SierraLeone']
```

Vamos observer as dez primeiras linhas de cálculo.

```
ebola_subset = ebola.loc[:,\
   ['Cases_Guinea','Cases_Liberia',\
      'Cases_SierraLeone','Cases_multiple']]
print(ebola_subset.head(n=10))
```

```
   Cases_Guinea  Cases_Liberia  Cases_SierraLeone  Cases_multiple
0        2776.0            NaN            10030.0             NaN
1        2775.0            NaN             9780.0             NaN
2        2769.0         8166.0             9722.0         20657.0
3           NaN         8157.0                NaN             NaN
4        2730.0         8115.0             9633.0         20478.0
5        2706.0         8018.0             9446.0         20170.0
6        2695.0            NaN             9409.0             NaN
7        2630.0         7977.0             9203.0         19810.0
8        2597.0            NaN             9004.0             NaN
9        2571.0         7862.0             8939.0         19372.0
```

Podemos ver que um valor para `Cases_multiple` foi calculado somente quando não havia valores ausentes para `Cases_Guinea`, `Cases_Liberia` e `Cases_SierraLeone`. Cálculos com valor ausente em geral devolverão um valor ausente, a menos que a função ou método chamado tenha um modo de ignorar os valores ausentes em seus cálculos.

Exemplos de métodos embutidos capazes de ignorar valores ausentes incluem `mean` e `sum`. Essas funções geralmente têm um parâmetro `skipna` permitindo que um valor ainda seja calculado, ignorando os valores ausentes.

```
# Ignorar valores ausentes é True por padrão
print(ebola.Cases_Guinea.sum(skipna=True))
```

```
84729.0
```

```
print(ebola.Cases_Guinea.sum(skipna=False))
```

```
nan
```

3.5 Conclusão dados ausentes

É raro ter um conjunto de dados sem qualquer valor ausente. É importante saber como trabalhar com valores ausentes porque, mesmo quando estiver trabalhando com dados completos, valores ausentes ainda poderão surgir durante a sua própria manipulação de dados.

Neste capítulo, analisamos alguns dos métodos básicos usados no processo de análise de dados pertinentes à validade dos dados. Ao observar seus dados e tabular os valores ausentes, podemos dar início ao processo de avaliar se os dados têm uma qualidade suficientemente boa para tomada de decisões e inferências.

4 *Tidy data* (dados organizados)

4.1 Introdução

Conforme mencionamos anteriormente, Hadley Wickham, um dos membros mais proeminentes da comunidade R, introduziu o conceito de *tidy data* (dados organizados) em um artigo do *Journal of Statistical Software*.

Tidy data é um framework para estruturar conjuntos de dados de modo que sejam facilmente analisados e visualizados. Podemos pensar nele como um objetivo a que devemos visar quando limpamos os dados.

Depois que você compreende o que é *tidy data*, esse conhecimento fará com que a análise, a visualização e a coleta de dados sejam muito mais fáceis.

Então o que é *tidy data*? O artigo de Hadley Wickham o define como um conceito que atende aos seguintes critérios:

- Cada linha é uma observação. (***observation***)
- Cada coluna é uma variável. (***variable***)
- Cada tipo de unidade de observação forma uma tabela.

Este capítulo descreve as diversas maneiras de organizar os dados conforme identificadas pelo artigo de Wickham.

Mapa conceitual

Conhecimento prévio:

a. Chamadas de função e de métodos;
b. Obtenção de subconjuntos de dados;
c. Laços;
d. `list comprehension` (abrangência de listas).

Neste capítulo:

- Refatoração de dados:
    a. `unpivot`/`melt`/`gather`("despivotear");
    b. `pivot`/`cast`/`spread`;
    c. Obtenção de subconjuntos;
    d. Combinação:
        1. Uso da biblioteca glob
        2. Concatenação

Objetivos

Este capítulo abordará:

1. Operação de `unpivoting`/`melting`/`gathering` de colunas em linhas ("despivotear");
2. Operação de `pivoting`/`casting`/`spreading` de linhas em colunas ("pivotear");
3. Normalização de dados separando um dataframe em várias tabelas;
4. Reunião de dados de várias partes.

4.2 Coluna contêm valores, e não variáveis

Os dados podem ter colunas que contêm valores em vez de variáveis. Em geral, é um formato conveniente para coleta e apresentação de dados.

4.2.1 Mantendo uma coluna fixa

Usaremos dados do **Pew Research Center** sobre renda e religião o Estados Unidos para mostrar como trabalhar com colunas que contenham **valores** em vez de **variáveis**.

```
# Importando dados Pew
pew = pd.read_csv("Data/Cap_06/pew.csv")
print(pew.head())
```

```
             religion  <$10k  $10-20k  $20-30k  $30-40k  $40-50k  $50-75k  \
0            Agnostic     27       34       60       81       76      137
1             Atheist     12       27       37       52       35       70
2            Buddhist     27       21       30       34       33       58
3            Catholic    418      617      732      670      638     1116
4  Don’t know/refused     15       14       15       11       10       35

   $75-100k  $100-150k  >150k  Don't know/refused
0       122        109     84                  96
1        73         59     74                  76
2        62         39     53                  54
3       949        792    633                1489
4        21         17     18                 116
```

Quando observamos esse conjunto de dados, podemos ver que nem todas as colunas são variável. Os valores relacionados à renda estão espalhados em várias colunas. O formato exibido é uma ótima opção quando apresentamos dados em uma tabela, mas, para análise de dados, a tabela precisa ser reformatada a fim de que tenhamos variáveis para religião, renda e contador.

```
# Exibe somente as primeiras colunas
print(pew.iloc[:,0:6])
```

```
                   religion  <$10k  $10-20k  $20-30k  $30-40k  $40-50k
0                  Agnostic     27       34       60       81       76
1                   Atheist     12       27       37       52       35
2                  Buddhist     27       21       30       34       33
3                  Catholic    418      617      732      670      638
4        Don't know/refused     15       14       15       11       10
5          Evangelical Prot    575      869     1064      982      881
6                     Hindu      1        9        7        9       11
7   Historically Black Prot    228      244      236      238      197
8         Jehovah's Witness     20       27       24       24       21
9                    Jewish     19       19       25       25       30
10            Mainline Prot    289      495      619      655      651
11                   Mormon     29       40       48       51       56
12                   Muslim      6        7        9       10        9
13                 Orthodox     13       17       23       32       32
14          Other Christian      9        7       11       13       13
15             Other Faiths     20       33       40       46       49
16    Other World Religions      5        2        3        4        2
17             Unaffiliated    217      299      374      365      341
```

Essa visualização dos dados também é conhecida como dados "largos" (*wide*). Para transformá-lo no formato de dados "longos" (*long*) do *tidy data*, teremos de efetuar uma operação de `unpivot`/`melt`/`gather` (depende da linguagem de programação estatística que estivermos usando) em nosso dataframe. O Pandas tem uma função chamada `melt` que reformatará o dataframe de maneira organizada.

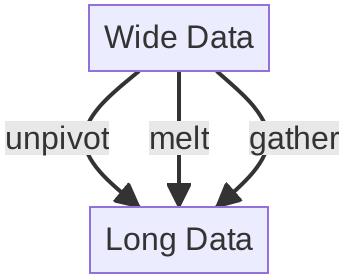

Figura 2: Transformando wide (largo) em long (longo) data.

`melt` aceita alguns **parâmetros**:

- `id_vars`:

  É um contêiner (lista, tupla, ndarray) que representa as variáveis que permanecerão inalteradas.

- `value_vars`:

  Identifica as colunas em que a operação de `melt` (ou `unpivot`) será executada. Por padrão, ela será executada em todas as colunas não especificadas no parâmetro `id_vars`.

- `var_name`:

  É uma string para o nome da nova coluna quando um `melt` é executado em `value_vars`. Por padrão, ela será chamada de `variable`.

- `value_name`:

  É uma string para o nome da nova coluna que presenta valores para `var_name`. Por padrão, ela será chamada de `value`.

```python
# Não precisamos especificar um value_vars, pois queremos pivotear
#todas as colunas, exceto a coluna 'religion'
pew_long = pd.melt(pew,id_vars='religion')

print(pew_long.head())
```

```
             religion variable  value
0            Agnostic    <$10k     27
1             Atheist    <$10k     12
2            Buddhist    <$10k     27
3            Catholic    <$10k    418
4  Don’t know/refused    <$10k     15
```

```python
print(pew_long.tail())
```

```
                  religion            variable  value
175               Orthodox  Don't know/refused     73
176        Other Christian  Don't know/refused     18
177           Other Faiths  Don't know/refused     71
178  Other World Religions  Don't know/refused      8
179           Unaffiliated  Don't know/refused    597
```

Podemos alterar os defaults de modo que as colunas sujeitas à operação de `melt/unpivot` sejam nomeadas.

```
pew_long = pd.melt(
   pew,
   id_vars='religion',
   var_name='income',
   value_name='count'
   )

print(pew_long.head())
```

```
             religion income  count
0            Agnostic  <$10k     27
1             Atheist  <$10k     12
2            Buddhist  <$10k     27
3            Catholic  <$10k    418
4  Don't know/refused  <$10k     15
```

```
print(pew_long.tail())
```

```
                  religion              income  count
175               Orthodox  Don't know/refused     73
176        Other Christian  Don't know/refused     18
177           Other Faiths  Don't know/refused     71
178  Other World Religions  Don't know/refused      8
179           Unaffiliated  Don't know/refused    597
```

4.2.2 Mantendo várias colunas fixas

Nem todo conjunto de dados terá uma coluna que permanecerá inalterada enquanto você executa um unpivot ("despivotear") no restante das colunas.

Como exemplo, considere o conjunto de dados Billboard:

```
billboard = pd.read_csv("Data/Cap_06/billboard.csv")

# Oberserve as primeiras linhas e colunas
print(billboard.iloc[0:5,0:16])
```

```
   year        artist                    track  time date.entered  wk1   wk2  \
0  2000         2 Pac  Baby Don't Cry (Keep...  4:22   2000-02-26   87  82.0
1  2000       2Ge+her  The Hardest Part Of ...  3:15   2000-09-02   91  87.0
2  2000  3 Doors Down               Kryptonite  3:53   2000-04-08   81  70.0
3  2000  3 Doors Down                    Loser  4:24   2000-10-21   76  76.0
4  2000      504 Boyz            Wobble Wobble  3:35   2000-04-15   57  34.0

    wk3   wk4   wk5   wk6   wk7   wk8   wk9  wk10  wk11
0  72.0  77.0  87.0  94.0  99.0   NaN   NaN   NaN   NaN
1  92.0   NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN
2  68.0  67.0  66.0  57.0  54.0  53.0  51.0  51.0  51.0
3  72.0  69.0  67.0  65.0  55.0  59.0  62.0  61.0  61.0
4  25.0  17.0  17.0  31.0  36.0  49.0  53.0  57.0  64.0
```

Podemos ver, nesse caso, que cada semana tem a própria coluna. Novamente, não há nada de errado com esse formato de dados. Talvez seja mais fácil inserir os dados nele; além disso, quando os dados são apresentados em uma tabela é muito mais rápido entender o seu significado.

No entento, pode haver um momento em que você precisará executar um `melt` nos dados. Por exemplo, se quisesse criar uma plotagem com faceta com as classificações semanais, a variável de facetar teria de ser uma coluna do dataframe.

```
billboard_long = pd.melt(
   billboard,
   id_vars=['year','artist','track','time','date.entered'],
   var_name='week',
   value_name='rating'
)

print(billboard_long.head())
```

```
   year        artist                    track  time date.entered week  rating
0  2000         2 Pac  Baby Don't Cry (Keep...  4:22   2000-02-26  wk1    87.0
1  2000       2Ge+her  The Hardest Part Of ...  3:15   2000-09-02  wk1    91.0
2  2000  3 Doors Down               Kryptonite  3:53   2000-04-08  wk1    81.0
3  2000  3 Doors Down                    Loser  4:24   2000-10-21  wk1    76.0
4  2000      504 Boyz            Wobble Wobble  3:35   2000-04-15  wk1    57.0
```

```
print(billboard_long.tail())
```

```
       year            artist                     track  time date.entered  \
24087  2000       Yankee Grey     Another Nine Minutes  3:10   2000-04-29
24088  2000  Yearwood, Trisha          Real Live Woman  3:55   2000-04-01
24089  2000   Ying Yang Twins  Whistle While You Tw...  4:19   2000-03-18
24090  2000     Zombie Nation            Kernkraft 400  3:30   2000-09-02
24091  2000   matchbox twenty                     Bent  4:12   2000-04-29

       week  rating
24087  wk76     NaN
24088  wk76     NaN
24089  wk76     NaN
24090  wk76     NaN
24091  wk76     NaN
```

4.3 Colunas contendo diversas variáveis

Às vezes, as colunas em um conjunto de dados podem representar diversas variáveis. Esse formato é comumente visto quando trabalhamos com dados de saúde, por exemplo.

Para ilustrar a situação, vamos observar o conjunto de dados Ebola.

```
ebola = pd.read_csv("Data/Cap_06/country_timeseries.csv")

print(ebola.columns)
```

```
Index(['Date', 'Day', 'Cases_Guinea', 'Cases_Liberia', 'Cases_SierraLeone',
       'Cases_Nigeria', 'Cases_Senegal', 'Cases_UnitedStates', 'Cases_Spain',
       'Cases_Mali', 'Deaths_Guinea', 'Deaths_Liberia', 'Deaths_SierraLeone',
       'Deaths_Nigeria', 'Deaths_Senegal', 'Deaths_UnitedStates',
       'Deaths_Spain', 'Deaths_Mali'],
      dtype='object')
```

```
# Exibe linhas selecionadas
print(ebola.iloc[:5,[0,1,2,3,10,11]])
```

```
         Date  Day  Cases_Guinea  Cases_Liberia  Deaths_Guinea  Deaths_Liberia
0    1/5/2015  289        2776.0            NaN         1786.0             NaN
1    1/4/2015  288        2775.0            NaN         1781.0             NaN
2    1/3/2015  287        2769.0         8166.0         1767.0          3496.0
3    1/2/2015  286           NaN         8157.0            NaN          3496.0
4  12/31/2014  284        2730.0         8115.0         1739.0          3471.0
```

Os nomes de colunas `Cases_Guinea` e `Deaths_Guinea` na verdade contêm duas variáveis: o status individual (casos e mortes, respectivamente), assim como o nome do país, Guinea (Guiné). Os dados também estão organizados em formato largo (`wide`), e a operação de `unpivot` (`melt`) deve ser executada.

```
ebola_long = pd.melt(
   ebola,
   id_vars=['Date','Day']
)

print(ebola_long.head())
```

```
         Date  Day      variable   value
0    1/5/2015  289  Cases_Guinea  2776.0
1    1/4/2015  288  Cases_Guinea  2775.0
2    1/3/2015  287  Cases_Guinea  2769.0
3    1/2/2015  286  Cases_Guinea     NaN
4  12/31/2014  284  Cases_Guinea  2730.0
```

```
print(ebola_long.tail())
```

```
           Date  Day     variable  value
1947  3/27/2014    5  Deaths_Mali    NaN
1948  3/26/2014    4  Deaths_Mali    NaN
1949  3/25/2014    3  Deaths_Mali    NaN
1950  3/24/2014    2  Deaths_Mali    NaN
1951  3/22/2014    0  Deaths_Mali    NaN
```

4.3.1 Separar e adicionar colunas individualmente (método simples)

Conceitualmente, a coluna de interesse pode ser separada com base no underscore no nome da coluna, "_". A primeira parte será a nova coluna de status, e a segunda será a nova coluna de país. Isso exigirá um pouco de parsing e separação de string em Python.

Em Python uma string é um objeto, semelhante ao modo como o Pandas tem objetos `Series` e `DataFrame`. As `Series` podem conter métodos como `mean`, e os `DataFrames` podem conter métodos como `to_csv`.

As strings também têm métodos. Nesse caso, usaremos o método `split`, que aceita uma string e a separa com base em um dado delimitador. Por padrão, `split` separará a string com base em um espaço, mas podemos passar o underscore, _, em nosso exemplo.

Para ter acesso aos métodos de string, temos de usar o método de acesso `str`. Isso nos dará acesso aos métodos de string de Python e nos permitirá trabalhar em toda a coluna.

```
# Obtém a coluna variável,
# acessa os métodos da string,
# e separa a coluna com base em um delimitador

variable_split = ebola_long.variable.str.split('_')

print(variable_split[:5])
```

```
0    [Cases, Guinea]
1    [Cases, Guinea]
2    [Cases, Guinea]
3    [Cases, Guinea]
4    [Cases, Guinea]
Name: variable, dtype: object
```

```
print(variable_split[-5:])
```

```
1947    [Deaths, Mali]
1948    [Deaths, Mali]
1949    [Deaths, Mali]
1950    [Deaths, Mali]
1951    [Deaths, Mali]
Name: variable, dtype: object
```

Depois da separação com base no underscore, os valores são devolvidos em uma lista. Sabemos se tratar de uma lista (`list`) porque é assim que o método `split` funciona, mas a pista visual encontra-se no fato de o resultado estar cercado por colchetes.

```
# O contêiner inteiro
print(type(variable_split))
```

```
<class 'pandas.core.series.Series'>
```

```
# O primeiro elemento do contêiner
print(type(variable_split[0]))
```

```
<class 'list'>
```

Agora que a coluna foi separada em várias partes, o próximo passo é atribuir essas partes a uma nova coluna. Inicialmente, porém, precisamos extrair todos os elementos com índice 0 da coluna de status e os elementos de índice 1 da coluna de país. Para isso, tamos de acessar os métodos de string novamente e então usar o método `get` para obter o índice que queremos em cada linha.

- O método `get()`:
  - **O que ele faz**:

    O `str.get(i)` acessa o elemento na posição i de cada lista dentro da Series.
  - **A lógica**:

    É como se você estivesse percorrendo a coluna e dizendo: "Pegue apenas o primeiro item de cada lista e coloque em uma nova coluna".

```
status_value = variable_split.str.get(0)
country_value = variable_split.str.get(1)

print(status_value[:5])
```

```
0    Cases
1    Cases
2    Cases
3    Cases
4    Cases
Name: variable, dtype: object
```

```
print(status_value[-5:])
```

```
1947    Deaths
1948    Deaths
1949    Deaths
1950    Deaths
1951    Deaths
Name: variable, dtype: object
```

```
print(country_value[:5])
```

```
0    Guinea
1    Guinea
2    Guinea
3    Guinea
4    Guinea
Name: variable, dtype: object
```

```
print(country_value[-5:])
```

```
1947    Mali
1948    Mali
1949    Mali
1950    Mali
1951    Mali
Name: variable, dtype: object
```

Agora que temos os vetores desejados, podemos adicioná-los em nosso dataframe.

```
ebola_long['status'] = status_value
ebola_long['country'] = country_value

print(ebola_long.head())
```

```
         Date  Day      variable   value status country
0    1/5/2015  289  Cases_Guinea  2776.0  Cases  Guinea
1    1/4/2015  288  Cases_Guinea  2775.0  Cases  Guinea
2    1/3/2015  287  Cases_Guinea  2769.0  Cases  Guinea
3    1/2/2015  286  Cases_Guinea     NaN  Cases  Guinea
4  12/31/2014  284  Cases_Guinea  2730.0  Cases  Guinea
```

4.3.2 Separar e combinar em um único passo (método simples)

Nesta seção, exploraremos o fato de que o vetor devolvido está na mesma ordem que nossos dados. Podemos concatenar o novo vetor aos nossos dados originais.

- O parâmetro `expand=True`

  Este parâmetro altera o tipo de objeto que o `split` devolve.

  - `expand=False` (Padrão):

    Retorna uma Series de listas.

    `Ex: 0    [Cases, Guinea]`

  - `expand=True`:

    Retorna um DataFrame completo, onde cada parte separada vira uma coluna própria.

    ```
    Ex:
    | | 0 | 1 |
    |---|---|---|
    | 0 | Cases | Guinea |
    ```

- `axis` parâmetro do `concat`:

  - `axis=1`:

    Adiciona colunas. Você está "colando" as novas informações (status e country) à direita da sua tabela original.

  - `axis=0` (**padrão**):

    Adicionaria linhas, uma embaixo da outra.

```
variable_split = ebola_long.variable.str.split('_',expand=True)
variable_split.columns = ['status','country']
ebola_parsed = pd.concat([ebola_long,variable_split], axis=1)

print(ebola_parsed.head())
```

```
         Date  Day      variable   value status country status country
0    1/5/2015  289  Cases_Guinea  2776.0  Cases  Guinea  Cases  Guinea
1    1/4/2015  288  Cases_Guinea  2775.0  Cases  Guinea  Cases  Guinea
2    1/3/2015  287  Cases_Guinea  2769.0  Cases  Guinea  Cases  Guinea
3    1/2/2015  286  Cases_Guinea     NaN  Cases  Guinea  Cases  Guinea
4  12/31/2014  284  Cases_Guinea  2730.0  Cases  Guinea  Cases  Guinea
```

```python
print(ebola_parsed.tail())
```

```
           Date  Day     variable  value  status country  status country
1947  3/27/2014    5  Deaths_Mali    NaN  Deaths    Mali  Deaths    Mali
1948  3/26/2014    4  Deaths_Mali    NaN  Deaths    Mali  Deaths    Mali
1949  3/25/2014    3  Deaths_Mali    NaN  Deaths    Mali  Deaths    Mali
1950  3/24/2014    2  Deaths_Mali    NaN  Deaths    Mali  Deaths    Mali
1951  3/22/2014    0  Deaths_Mali    NaN  Deaths    Mali  Deaths    Mali
```

Explicando o código:

- Pega a coluna variable (que tem textos como `Cases_Guinea`), corta onde tem o sublinhado (_) e cria duas colunas separadas graças ao `expand=True`.
- Define que essas duas novas colunas agora se chamam oficialmente `status` e `country`.
- Usa o `pd.concat` com `axis=1` para pegar essas duas novas colunas e grudá-las ao lado da sua tabela original (`ebola_long`).

4.3.3 Separar e combinar em um único passo (método mais complicado)

Nesta subseção, exploraremos novamente o fato de que o vetor devolvido está na mesma ordem que nossos dados. Podemos concatenar o novo vetor aos nossos dados originais.

É possível obter o mesmo resultado com um único passo tirando proveito do fato de que o resultado da separação devolve uma lista com dois elementos, em que cada elemento e uma nova coluna.

Podemos combinar a lista de itens separados com a função embutida `zip`, `zip` aceita um conjunto de iteradores (por exemplo, listas, tuplas) e cria um contêiner composto dos iteradores de entrada, mas cada novo contêiner criado tem o mesmo índice que os contêineres de entrada.

Por exemplo, se tivermos duas listas de valores,

```
constants = ['pi','e']
values = ['3.14','2.718']
```

podemos executar `zip` nas duas:

```
# Temos que chamar list na função zip
# para exibir o conteúdo do objeto zip;
# Em Python 3, zip devolve um iterador
print(list(zip(constants,values)))
```

```
[('pi', '3.14'), ('e', '2.718')]
```

Cada elemento agora tem a constante correspondente ao seu valor.

Conceitualmente, cada contêiner é como um lado de um ziper. Quando executamos `zip` nos contêineres, há uma correspondência entre os índices, e esses dados são devolvidos.

Outra forma de visualizar o que o `zip` faz é tomar cada contêiner passado para ele e empilhá-los uns sobre os outros (pense na concatenação por linhas), criando, assim, uma espécie de dataframe. `zip` então devolve os valores coluna a coluna em uma tupla.

Podemos usar o mesmo `ebola_long.variable.str.split('_')` para separar os valores da coluna. No entanto, como o resultado já é um contêiner (um objeto `Series`), temos que desempacotá-lo para obter o seu conteúdo (cada lista status-país), em vez de termos o próprio contêiner (a série).

Em Python, o operador asterisco, *, é usado para desempacotar contêineres. Quando executamos `zip` nos contêineres desempacotados, o efeito será o mesmo obtido quando criamos os valores de status e de país anteriormente. Podemos, então, atribuir os vetores às colunas simultaneamente usando a atribuição múltipla.

```
ebola_long['status'], ebola_long['country'] = \
   zip(*ebola_long.variable.str.split('_'))

print(ebola_long.head())
```

```
         Date  Day      variable   value status country
0    1/5/2015  289  Cases_Guinea  2776.0  Cases  Guinea
1    1/4/2015  288  Cases_Guinea  2775.0  Cases  Guinea
2    1/3/2015  287  Cases_Guinea  2769.0  Cases  Guinea
3    1/2/2015  286  Cases_Guinea     NaN  Cases  Guinea
4  12/31/2014  284  Cases_Guinea  2730.0  Cases  Guinea
```

A lógica funciona assim:

- `split('_')`: Quebra a string (ex: "Cases_Guinea") em uma lista: `['Cases', 'Guinea']`.
- O Asterisco (`*`): Ele "abre a caixa". Em vez de passar a coluna inteira para o `zip`, ele passa cada lista individualmente.
- O `zip`: Ele funciona como um zíper de jaqueta. Ele pega o primeiro item de cada lista e agrupa (todos os "Status"), depois pega o segundo item de cada lista e agrupa (todos os "Países").

Aprofundando a função `zip`:

Imagine que o `zip` é um grampeador de listas. Ele pega o primeiro item de cada lista e grampeia juntos, depois o segundo, e assim por diante.

Exemplo simples:

```
nomes = ['Ana', 'Beto']
idades = [25, 30]

# O zip vai unir o 1º com o 1º, e o 2º com o 2º
resultado = list(zip(nomes, idades))

print(resultado)
```

```
[('Ana', 25), ('Beto', 30)]
```

Por que o texto usa o asterisco `zip(*...)`?

- No seu caso do Pandas, você não tem duas listas separadas no início. Você tem uma única coluna onde cada linha é uma lista, tipo assim:

`[['Cases', 'Guinea'], ['Deaths', 'Guinea']].`

- Se você der o `zip` direto nessa coluna, ele não sabe o que fazer. O asterisco (*) "derrama" o conteúdo dessa coluna.
- Imagine que o asterisco faz isso:

`zip(['Cases', 'Guinea'], ['Deaths', 'Guinea'])`

- O zip então "grampeia":
  - Pega todos os primeiros itens (índice 0): ('Cases', 'Deaths') -> Vai para a coluna Status.
  - Pega todos os segundos itens (índice 1): ('Guinea', 'Guinea') -> Vai para a coluna Country.

4.4 Variáveis tanto em linhas quanto em colunas

As vezes, os dados estarão formatados de modo que as variáveis se encontrarão tanto nas linhas quanto nas colunas - isto é, em alguma combinação de formatos descritos nas seções anteriores deste capítulo. A maioria dos métodos necessários para organizar dados desse tipo já foi apresentada. O que resta mostrar é o que acontecerá se uma coluna de dados armazenar duas variáveis em vez de uma. Nesse caso, teremos de executar uma operação de `pivot` ou `cast` da variável em colunas separadas.

```
weather = pd.read_csv("Data/Cap_06/weather.csv")

# Dataframe no formato largo (wide)
print(weather.iloc[:5,:11])
```

```
        id  year  month element  d1    d2    d3  d4    d5  d6  d7
0  MX17004  2010      1    tmax NaN   NaN   NaN NaN   NaN NaN NaN
1  MX17004  2010      1    tmin NaN   NaN   NaN NaN   NaN NaN NaN
2  MX17004  2010      2    tmax NaN  27.3  24.1 NaN   NaN NaN NaN
3  MX17004  2010      2    tmin NaN  14.4  14.4 NaN   NaN NaN NaN
4  MX17004  2010      3    tmax NaN   NaN   NaN NaN  32.1 NaN NaN
```

Os dados meteorológicos incluem temperaturas mínima e máxima (valores `tmin` e `tmax` na coluna `element`, respectivamente) registradas para cada dia (`d1`, `d2`, `d3`,..., `d31`) do mês (`month`).

A coluna element contém variáveis que devem ser submetidas a `cast`/`pivot` para que se transformem em novas colunas, e as variáveis de dia devem ser sujeitas à operação de `melt`, gerando valores de linha.

Novamente, não há nada de errado com os dados no formato atual. Eles simplesmente não estão em um formato apropriado para análises, embora esse tipo de formatação possa ser útil quando apresentamos dados em relatórios.

Vamos executar primeiro a operação de `melt`/`unpivot` nos valores dos dias.

```
weather_melt = pd.melt(
   weather,
   id_vars=['id','year','month','element'],
   var_name='day',
   value_name='temp'
)

print(weather_melt.head())
```

```
        id  year  month element day  temp
0  MX17004  2010      1    tmax  d1   NaN
1  MX17004  2010      1    tmin  d1   NaN
2  MX17004  2010      2    tmax  d1   NaN
3  MX17004  2010      2    tmin  d1   NaN
4  MX17004  2010      3    tmax  d1   NaN
```

```
print(weather_melt.tail())
```

```
          id  year  month element  day  temp
677  MX17004  2010     10    tmin  d31   NaN
678  MX17004  2010     11    tmax  d31   NaN
679  MX17004  2010     11    tmin  d31   NaN
680  MX17004  2010     12    tmax  d31   NaN
681  MX17004  2010     12    tmin  d31   NaN
```

A seguir, devemos pivotear as variáveis armazenadas na coluna `element`. Esse processo é conhecido como *casting* ou *spreading* em outras linguagens estatística.

Uma das principais diferenças entre `pivot_table` e `melt` é que `melt` é uma função no Pandas, enquanto pivot_table é um método que chamamos em um objeto `DataFrame`.

```
weather_tidy = weather_melt.pivot_table(
   index=['id','year','month','day'],
   columns='element',
   values='temp'
)

print(weather_tidy.head())
```

```
element                 tmax  tmin
id      year month day
MX17004 2010 1     d30  27.8  14.5
             2     d11  29.7  13.4
                   d2   27.3  14.4
                   d23  29.9  10.7
                   d3   24.1  14.4
```

Sobre os parâmetros do `pivot_tabel`:

- `index`: Define quais colunas não devem mudar. Elas identificam a linha (neste caso: `id`, `year`, `month`, `day`).
- `columns`: Indica qual coluna contém os nomes que você quer transformar em novas colunas. No texto, é a coluna element (que contém os textos `tmax` e `tmin`).
- `values`: Indica de onde virão os números (os dados reais) para preencher essas novas colunas. No exemplo, os valores vêm da coluna `temp`.

Observando a tabela após a operação de pivot, percebemos que cada valor na coluna element está agora em uma coluna separada. É possível deixar essa tabela em seu estado atual, mas também podemos linearizar as colunas hierárquicas.

```
weather_tidy_flat = weather_tidy.reset_index()
print(weather_tidy_flat.head())
```

```
element       id  year  month  day  tmax  tmin
0        MX17004  2010      1  d30  27.8  14.5
1        MX17004  2010      2  d11  29.7  13.4
```

```
2        MX17004  2010      2   d2  27.3  14.4
3        MX17004  2010      2  d23  29.9  10.7
4        MX17004  2010      2   d3  24.1  14.4
```

Quando você executa o `pivot_table`, o Pandas transforma as colunas que você passou no parâmetro `index` (como `id`, `year`, `month`, `day`) em um `MultiIndex` (índices hierárquicos). Visualmente, a tabela fica com as linhas "agrupadas" por essas categorias, o que pode dificultar manipulações posteriores.

O `.reset_index()` serve para:

- Linearizar os dados: Ele retira essas colunas da posição de "índice" e as devolve para o corpo principal da tabela como colunas comuns.
- Remover a hierarquia: Ele cria um novo índice numérico padrão (0, 1, 2…).
- Facilitar o acesso: Torna a tabela "plata" (flat), que é o formato padrão do Tidy Data, facilitando filtros e exportações (como para Excel ou CSV).

Do mesmo modo, podemos aplicar esses métodos sem o dataframe intermediário:

```
weather_tidy = weather_melt.\
   pivot_table(
      index=['id','year','month','day'],
      columns='element',
      values='temp'
   ).\
   reset_index()

print(weather_tidy.head())
```

```
element       id  year  month  day  tmax  tmin
0        MX17004  2010      1  d30  27.8  14.5
1        MX17004  2010      2  d11  29.7  13.4
2        MX17004  2010      2   d2  27.3  14.4
3        MX17004  2010      2  d23  29.9  10.7
4        MX17004  2010      2   d3  24.1  14.4
```

O Problema do "Destaque" (O Índice)

Quando você usa o `pivot_table`, o Pandas decide que as colunas que você usou no parâmetro `index` (como `id`, `year`, `month`, `day`) são tão importantes que elas viram "etiquetas de linha".

Visualmente, a tabela fica assim:

- As colunas id, ano, mês, dia ficam em negrito e em um nível diferente.
- Elas não se comportam mais como colunas normais onde você pode fazer cálculos; elas viram apenas "nomes das linhas".

Isso é o que o texto chama de hierarquia. É ótimo para um relatório visual (fica bonito de ler), mas é péssimo para o computador processar depois.

A Solução: `reset_index()` (A Reindexação)

O `reset_index()` é como se você desse um comando de "voltem para o seu lugar!".

- Transforma etiquetas em colunas: Ele pega aquelas informações que estavam "presas" no índice (em negrito/destaque) e as coloca de volta como colunas normais da tabela.
- Cria um contador simples: No lugar delas, ele cria um índice novo, que é apenas uma contagem simples: 0, 1, 2, 3…

4.5 Várias unidades de observação em uma tabela (normalização)

Um dos modos mais simples de saber se várias unidades de observação estão representadas em uma tabela é observar cada um das linhas e prestar atenção se há alguma célula ou valor sendo repetido nas linhas.

Isso é muito comum em dados administrativos de educação do governo, em que informações demográficas dos estudantes são informações para cada aluno, para cada ano em que esse aluno estiver matriculado.

Vamos observar novamente os dados da Billboard que limpamos anteriormente.

```
print(billboard_long.head())
```

```
   year        artist                    track  time date.entered week  rating
0  2000         2 Pac  Baby Don't Cry (Keep...  4:22   2000-02-26  wk1    87.0
1  2000       2Ge+her  The Hardest Part Of ...  3:15   2000-09-02  wk1    91.0
2  2000  3 Doors Down               Kryptonite  3:53   2000-04-08  wk1    81.0
3  2000  3 Doors Down                    Loser  4:24   2000-10-21  wk1    76.0
4  2000      504 Boyz            Wobble Wobble  3:35   2000-04-15  wk1    57.0
```

Supanha que tenhamos criado um subconjunto de dados com base em uma faixa musical em particular.

```
print(billboard_long[billboard_long.track == 'Loser'].head())
```

```
      year        artist  track  time date.entered week  rating
3     2000  3 Doors Down  Loser  4:24   2000-10-21  wk1    76.0
320   2000  3 Doors Down  Loser  4:24   2000-10-21  wk2    76.0
637   2000  3 Doors Down  Loser  4:24   2000-10-21  wk3    72.0
954   2000  3 Doors Down  Loser  4:24   2000-10-21  wk4    69.0
1271  2000  3 Doors Down  Loser  4:24   2000-10-21  wk5    67.0
```

- Análise do código:

  Ele realiza uma filtragem no DataFrame original (`billboard_long`).

  - `billboard_long.track == 'Loser'`: Procura na coluna de músicas (`track`) apenas as linhas onde o nome é 'Loser'.
  - `billboard_long[...]`: Seleciona essas linhas específicas.
  - `.head()`: Mostra apenas as 5 primeiras ocorrências desse filtro.

- Por que ele é importante para o `Tidy Data`?

  Ao isolar uma única música, fica visualmente claro o problema da redundância:

  - Você verá que o nome do artista, o nome da faixa e o tempo da música se repetem identicamente em todas as linhas.
  - A única coisa que realmente muda de linha para linha é a semana e a classificação (rating).

Podemos ver que essa tabela, na verdade, armazena dois tipos de dados: a informação da faixa musical (`track`) e a classificação semanal.

Seria melhor armazenar a informação da faixa em uma tabela separada. Dessa forma, as informações armazenadas nas colunas `year`, `artist`, `track` e `time` não estariam repetidas no conjunto de dados. Essa consideração é particularmente importante se os dados forem inseridos manualmente. Repetir os mesmos valores de modo contínuo durante a entrada de dados eleva o risco de haver dados inconsistentes.

O que deveríamos fazer nesse caso é colocar `year`, `artist`, `track`, `time` e `date.entered` em um novo dataframe, com cada conjunto único de valores recebendo um `ID` único.

Podemos usar esse `ID` único em um segundo dataframe que represente a música, a data, o número da semana e a classificação.

Esse processo como um todo pode ser pensado como uma inversão dos passos de concatenação e combinação de dados.

```
billboard_song = billboard_long[['year','artist','track','time']]
print(billboard_song.head())
```

```
   year        artist                    track  time
0  2000         2 Pac  Baby Don't Cry (Keep...  4:22
1  2000       2Ge+her  The Hardest Part Of ...  3:15
2  2000  3 Doors Down               Kryptonite  3:53
3  2000  3 Doors Down                    Loser  4:24
4  2000      504 Boyz            Wobble Wobble  3:35
```

```
print(billboard_song.shape)
```

```
(24092, 4)
```

- Análise do código:
  - O código cria um novo DataFrame chamado `billboard_song` selecionando apenas as colunas que descrevem a música (`year`, `artist`, `track`, `time`).

Sabemos que há entradas duplicadas nesse dataframe, portanto devemos descartar as linhas duplicadas.

```
billboard_song = billboard_song.drop_duplicates()
print(billboard_song.head())
```

```
   year        artist                    track  time
0  2000         2 Pac  Baby Don't Cry (Keep...  4:22
1  2000       2Ge+her  The Hardest Part Of ...  3:15
2  2000  3 Doors Down               Kryptonite  3:53
3  2000  3 Doors Down                    Loser  4:24
4  2000      504 Boyz            Wobble Wobble  3:35
```

```python
print(billboard_song.shape)
```

```
(317, 4)
```

- Análise do código:
  - `drop_duplicates()`: Varre o DataFrame `billboard_song` e remove todas as linhas que são exatamente iguais a uma linha anterior.
  - Resultado: Se a música "Loser" aparecia em 20 linhas (uma para cada semana), agora ela aparece em apenas uma linha.

Podemos então atribuir um valor único para cada linha de dados.

```
billboard_song['id'] = range(len(billboard_song))
print(billboard_song.head(n=10))
```

```
   year          artist                    track  time  id
0  2000           2 Pac  Baby Don't Cry (Keep...  4:22   0
1  2000         2Ge+her  The Hardest Part Of ...  3:15   1
2  2000    3 Doors Down               Kryptonite  3:53   2
3  2000    3 Doors Down                    Loser  4:24   3
4  2000        504 Boyz            Wobble Wobble  3:35   4
5  2000            98^0  Give Me Just One Nig...  3:24   5
6  2000         A*Teens            Dancing Queen  3:44   6
7  2000         Aaliyah            I Don't Wanna  4:15   7
8  2000         Aaliyah                Try Again  4:03   8
9  2000  Adams, Yolanda            Open My Heart  5:30   9
```

- Análise do código:
  - `len(billboard_song)`: Conta quantas músicas únicas restaram após o `drop_duplicates`.
  - `range(...)`: Gera uma sequência numérica (0, 1, 2…) do mesmo tamanho da tabela.
  - `billboard_song['id']`: Cria uma nova coluna chamada id e atribui esses números a cada linha.
- Por que isso é crucial?
  - Chave Primária: Esse id agora é o "RG" de cada música.
  - Eficiência: Em vez de repetir "Artist: Beck, Track: Loser, Time: 3:55" em centenas de linhas na outra tabela, você usará apenas o número do `id`.
  - Organização Mental: Isso separa claramente quem é o objeto (a música) de qual é o evento (a classificação na Billboard).

Agora que temos um dataframe separada com as músicas, podemos usar a coluna `id` recém-criada para fazer uma correspondência entre uma música e sua classificação semanal.

```
# Combina o dataframe de música com o conjunto de dados original
billboard_ratings = billboard_long.merge(
   billboard_song,
   on=['year','artist','track','time']
   )
print(billboard_ratings.head())
```

```
   year        artist                    track  time date.entered week  \
0  2000         2 Pac  Baby Don't Cry (Keep...  4:22   2000-02-26  wk1
1  2000       2Ge+her  The Hardest Part Of ...  3:15   2000-09-02  wk1
2  2000  3 Doors Down               Kryptonite  3:53   2000-04-08  wk1
3  2000  3 Doors Down                    Loser  4:24   2000-10-21  wk1
4  2000      504 Boyz            Wobble Wobble  3:35   2000-04-15  wk1

   rating  id
0    87.0   0
1    91.0   1
2    81.0   2
3    76.0   3
4    57.0   4
```

```
print(billboard_ratings.shape)
```

```
(24092, 8)
```

- O que acontece aqui:
  - O Pandas olha para a tabela original (`billboard_long`) e para a nova tabela de músicas (`billboard_song`) e diz: "Onde os valores de `year`, `artist`, `track` e `time` forem iguais nas duas tabelas, traga a coluna `id` para cá".

Por fim, vamos obter um subconjunto com as colunas que queremos em nosso dataframe de classificações.

```
billboard_ratings = \
   billboard_ratings[['id','date.entered','week','rating']]
print(billboard_ratings.head())
```

```
   id date.entered week  rating
0   0   2000-02-26  wk1    87.0
1   1   2000-09-02  wk1    91.0
2   2   2000-04-08  wk1    81.0
3   3   2000-10-21  wk1    76.0
4   4   2000-04-15  wk1    57.0
```

```
print(billboard_ratings.shape)
```

```
(24092, 4)
```

- Análise do código:
  - Filtragem de Colunas: Ele descarta as colunas textuais (`artist`, `track`, `time`) que agora estão seguras na outra tabela (`billboard_song`).
  - Referência por ID: Mantém apenas o id (o "RG" da música) para sabermos a qual faixa aquele rating se refere.
- Resultado Final:
  - Tabela 1 (`billboard_song`): Contém os dados fixos da música (Cadastro).
  - Tabela 2 (`billboard_ratings`): Contém o desempenho semanal (Transações).

Revisão normalização de tabelas banco de dados

As regras de normalização (Formas Normais) visam eliminar a redundância e proteger a integridade dos dados.

As três principais são:

1ª Forma Normal (**1FN**)

- Regra: Eliminar grupos repetidos e garantir que cada campo contenha apenas valores atômicos (únicos).
- Ação: Cada interseção de linha e coluna deve ter apenas um dado.
- Exemplo: Se na coluna telefones você tem (11) 999, (11) 888, isso viola a **1FN**. Você deve separar em duas linhas ou duas colunas.

2ª Forma Normal (**2FN**)

- Regra: Estar na **1FN** e garantir que todos os atributos dependam da chave primária total.
- Ação: Eliminar dependências parciais (comum em chaves compostas).
- Exemplo: Na tabela original, o `artist` dependia da música, mas não dependia da `week` (semana). Ao separar a música em uma tabela própria com seu próprio `ID`, garantimos que o artista esteja "amarrado" apenas ao que lhe cabe.

3ª Forma Normal (**3FN**)

- Regra: Estar na **2FN** e garantir que nenhum atributo dependa de outro atributo que não seja a chave.
- Ação: Eliminar dependências transitivas (um campo não-chave determinando outro campo não-chave).
- Exemplo: Se você tem as colunas `cidade` e `estado`. O `estado` depende da `cidade`, e não diretamente do `ID` do usuário. Você criaria uma tabela de "Cidades" para não ter que repetir o estado toda vez que escrever o nome da cidade.

Tabela 5: Comparando Normalização SQL e Tidy Data

Conceito	Foco Principal	Conceito Comum
Normalização	Integridade e economia de espaço (evita redundância).	Bancos de Dados Relacionais (SQL).
Tidy Data	Facilidade de análise e visualização.	Ciência de Dados (R, Python, Excel).

4.6 Unidade de observação em várias tabelas

A última porção da organização dos dados relaciona-se à situação em que o mesmo tipo de dado está espalhado por várias tabelas. Esse problema já foi abordado anteriormente, em que discutimos a concatenação e o `merge` de dados.

Um dos motivos pelos quais os dados podem estar separados em vários arquivos é o tamanho desses arquivos. Ao separar dados em várias partes, cada uma delas se torna menor. Isso pode ser bom se houver necessidade de compartilhar dados na internet ou por email, pois muito serviços limitam o tamanho de um arquivo que poder ser aberto ou compartilhado. O processo de coleta de dados é outro motivo pelo qual um conjunto de dados pode estar separado em várias partes. Por exemplo, um conjunto de dados separado contendo informações sobre ações no mercado poderia ser criado para cada dia.

Como o `merge` e a concatenação já foram discutidos, esta seção terá como foco as técnicas para carregar rapidamente várias fontes de dados e reuni-las.

Exemplo:

O Unified New York City Taxi and Uber Data (Dados unificados de táxi e Uber da cidade de Nova York) é uma boa opção para demonstrar esses processos.

O conjunto completo contém dados sobre mais de 1,3 Bilhão de corridas de táxi e de Uber na cidade de Nova York e esta organizado em mais de 140 arquivos.

Como demonstração, trabalharemos com apenas cinco desses arquivos de dados.

Quando os mesmos dados estão separados em várias partes, em geral, estas tem um padrão de nomeclatura estruturado associado a elas.

Em primeiro lugar, vamos fazer download dos dados. Não se preocupe muito com os detalhes do bloco de código a seguir. O arquivo `raw_data_urls.txt` contém uma lista de URLs, em que cada URL é o link de download para uma parte dos dados de táxi.

Começamos abrindo e lendo o arquivo e iterando pelas linhas do arquivo (isto é, cada URL de dados). Faremos o download somente dos cinco primeiros conjuntos de dados, pois os arquivos são bem grandes. Usaremos um pouco de manipulação de strings para criar o path em que os dados serão salvos e utilizaremos a biblioteca `urllib` para fazer o download de nossos dados.

Bibliotecas usadas:

- `urllib`

  A urllib é o canivete suíço nativo do Python para lidar com URLs. Ela permite que seu código interaja com a internet sem precisar de navegadores.

  - O que faz: Abre, lê e manipula dados de links da web.
  - Principais usos:
    * Fazer requisições HTTP (baixar o conteúdo de uma página ou API).
    * Codificar parâmetros de busca (transformar espaços em %20, por exemplo).
    * Analisar a estrutura de uma URL (separar o domínio do caminho).
  - Ponto importante: Embora seja muito poderosa e nativa, muitos desenvolvedores preferem a biblioteca externa `requests` por ser mais "amigável". Mas a `urllib` é quem manda nos bastidores.

- `glob`

  A `glob` é o seu "detetive de arquivos". Ela serve para encontrar caminhos de arquivos e pastas seguindo padrões específicos, de forma muito mais simples do que listar tudo manualmente.

  - O que faz: Retorna uma lista de arquivos que correspondem a um padrão (usando wildcards ou "coringas").
  - Principais usos:
    * Listar todos os arquivos `.jpg` de uma pasta específica.
    * Encontrar arquivos que começam com "relatorio_" e terminam em .pdf.
    * Navegar por subdiretórios rapidamente.
  - Exemplo de padrão: `glob.glob("*.csv")` traria todos os arquivos Excel/CSV da pasta atual.

```
import os
import urllib.request

# 1. Ajustamos o caminho base usando o expanduser para o Ubuntu entender o '~'
base_path = os.path.expanduser("~/Programacao/python_dados/Data/Cap_06/varias_tabelas")

# 2. Caminho completo do arquivo TXT
arquivo_urls = os.path.join(base_path, "raw_data_urls.txt")

# 3. Abrimos o arquivo usando o caminho completo
with open(arquivo_urls, 'r') as data_urls:
    for line, url in enumerate(data_urls):
        if line == 5:
            break

        # Extrai o nome do arquivo (ex: tabela1.csv)
        fn = url.split('/')[-1]

        # Define onde salvar: EXATAMENTE na mesma pasta do TXT
        fp = os.path.join(base_path, fn)

        print(f"Baixando linha {line}: {url}")
        print(f"Salvando em: {fp}")

        # Realiza o download
        urllib.request.urlretrieve(url, fp)
```

Infelizmente os links estão quebrados. Esse código não funciona mais, mesmo adaptando. Essa parte será puramente teórica.

Explicando o código:

- `with open(arquivo_urls, 'r') as data_urls:`

Abre o seu arquivo .txt. o `with` é um "segurança". Ele garante que, assim que o código terminar de ler as URLs (ou se der algum erro no meio), o arquivo seja fechado automaticamente. Sem o `with`, o arquivo poderia ficar "preso" na memória do Ubuntu.

- `for line, url in enumerate(data_urls):`

Normalmente, um for leria apenas a url. O `enumerate` adiciona um contador automático. Na primeira volta, `line` vale 0, na segunda 1, e assim por diante. É por isso que você consegue usar o `if line == 5: break` para parar o download após 5 arquivos.

- `fn = url.split('/')[-1]`
  - O `.split('/')` corta a frase em todos os lugares que tem uma barra, transformando-a em uma lista: `['https:', '', 'site.com', 'dados', 'tabela1.csv']`.
  - O `[-1]` pega o último item dessa lista.
  - Resultado: `fn` (filename) passa a ser apenas `tabela1.csv`.
- `fp = os.path.join(base_path, fn)`
  - `fp` significa File Path (caminho do arquivo).
  - Em vez de você escrever manualmente `/home/sergio/.../tabela1.csv`, o Python junta o caminho da pasta (`base_path`) com o nome do arquivo (`fn`) de forma limpa.
- `urllib.request.urlretrieve(url, fp)`
  - Esta é a função que efetivamente "teletransporta" o arquivo da web para o seu HD.
  - Ela precisa de dois argumentos:
    * De onde vem? (`url`)
    * Para onde vai? (`fp`).

Nesse exemplo, todos os arquivos com dados brutos das corridas de táxi têm o padrão `fhv_tripdata_YYYY_XX.csv`, em que `YYYY` representa o ano (por exemplo, 2015) e `XX` representa o número da parte. Podemos usaruma função simples de correspondência de padrões da biblioteca `glob` em Python para obter uma lista com todos os nomes de arquivos que correspondam a um determinado padrão.

```
import glob

#Obtém uma lista de arquivos csv da pasta de dados das corridas de táxi de nyc
nyc_taxi_data = glob.glob('../data/fhv_*')
print(nyc_taxi_data)
```

Agora que temos uma lista dos nomes dos arquivos que queremos carregar, podemos carregar cada arquivo emum dataframe. É possível optar por corregar cada arquivo individualmente, como temos feito até agora.

```
taxi1 = pd.read_csv(nyc_taxi_data[0])
taxi2 = pd.read_csv(nyc_taxi_data[1])
taxi3 = pd.read_csv(nyc_taxi_data[2])
taxi4 = pd.read_csv(nyc_taxi_data[3])
taxi5 = pd.read_csv(nyc_taxi_data[4])
```

Podemos observar nossos dados e ver como é possível empilhá-los (concatená-los) elegantemente uns sobre os outros.

```
print(taxi1.head(n=2))
print(taxi2.head(n=2))
print(taxi3.head(n=2))
print(taxi4.head(n=2))
print(taxi5.head(n=2))
```

Os dados podem ser concatenados exatamente como fizemos em capítulos anteriores.

```
# Formato de cada dataframe
print(taxi1.shape)
print(taxi2.shape)
print(taxi3.shape)
print(taxi4.shape)
print(taxi5.shape)
```

```
# Concatena os dataframes
taxi = pd.concat([taxi1,taxi2,taxi3,taxi4,taxi5])

# Formato final dos dados de táxi concatenados
print(taxi.shape)
```

Entretanto, salvar manualmente cada dataframe se tornará tedioso quando os dados estiverem divididos em muitas partes. Como uma abordagem alternativa, podemos automatizar o processo usando laços e `list comprehensions` (abrangência de listas).

4.6.1 Carregando vários arquivos usando um laço

4.6.2 Carregando vários arquivos usando uma `list comprehension`

4.7 Conclusão *Tidy data*

Tabela 6: Métodos para transformação largo (wide) e longo (long)

Recurso	`pd.melt()`	`.pivot_table()`
Objetivo	Transformar colunas em linhas (**Unpivot**).	Transformar linhas em colunas (**Pivot**).
Direção	De Largo (Wide) para Longo (Long).	De Longo (Long) para Largo (Wide).
Tidy Data	Formato ideal para análise e gráficos.	Formato ideal para relatórios e tabelas dinâmicas.
Resultado	Aumenta o número de linhas.	Reduz o número de linhas e cria novos cabeçalhos.
Sintaxe	Função do pandas (`pd.melt(df)`)	Método do DataFrame (`df.pivot_table()`)